Hydrofrac user guide

Hydrofrac tests whether a Hydro program keeps its eventual-consistency promises when nodes crash and messages are lost. It runs the program in the Hydro simulator, injects faults, and looks for a run that ends with two surviving replicas holding different data. If it finds one, it reports the faults. If it does not, it reports how much of the fault space it covered.

The guide has four parts:

1. The problem
2. How hydrofrac works
3. How to use it, with every example in the test corpus
4. Limitations and open questions

All outputs in this guide were captured by running the tests on the current tree, except where a section says otherwise.

1. The problem

Hydro lets a program mark a collection on a cluster as `EventualConsistency` (EC). The mark promises that all replicas end up with the same contents.

That promise has two halves:

1. **Same inputs, same result.** Two replicas that receive the same messages compute the same contents, in any arrival order.
2. **Same inputs.** Faults do not leave surviving replicas with permanently different inputs.

Hydro's type system checks the first half. It knows which operators are insensitive to order and duplicates, and it drops the label after any operator that is not.

The type system cannot check the second half. Whether every survivor gets every input depends on the protocol's redundancy: does anyone resend, echo, or gossip a message after its sender dies? That is a property of the protocol's behavior under faults, not of its types. So without hydrofrac, an EC label is only half-checked.

A small example. A process sends two items to three replicas over `broadcast_closed`, which sends each item to each replica in turn. Each replica keeps the set of items it received. Set insertion is order-insensitive, so the type system grants the label. Now the sender crashes after sending item 2 to replicas 0 and 1, but before sending it to replica 2. Nothing in the program sends item 2 again. Replicas 0 and 1 hold `{1, 2}`, replica 2 holds `{1}`, forever.

Hydrofrac checks the second half. For each collection labelled EC, it searches for faults that leave the survivors with different contents.

2. How hydrofrac works

Hydrofrac implements lineage-driven fault injection (LDFI), the technique introduced by the Molly tool (Alvaro, Rosen and Hellerstein, SIGMOD 2015). LDFI works backwards from a successful run:

1. Run the program without faults and record which message deliveries produced the result.
2. Choose faults that remove at least one of those deliveries.
3. Run again with those faults. If the result still holds, the program found another route. Record it and go back to step 2, now removing a delivery from every recorded route.
4. Stop when a fault set breaks the result (a counterexample), or when no allowed fault set can remove every recorded route (a certificate).

Compared with Molly, hydrofrac makes some things simpler:

- It runs the real Hydro program in the simulator. Molly needed the program rewritten in a logic language.
- It puts a fault point on every network channel. The test does not list them.
- It generates the check from the EC labels. The test does not write a specification.
- It names faults by messages, not by logical clock times.

It also adds some things:

- It explores message interleavings under each fault set. Molly ran one.
- It records lineage at two levels of detail.
- It knows when one interleaving per fault set is enough.

The rest of this section covers each piece.

2.1 Faults

The simulator compiles the whole distributed program into one process. Network channels become queues. On the sending side of every channel, the generated code calls a filter before a message enters the queue (`record_sent` in `hydro_lang/src/sim/runtime/lineage.rs`, generated in `hydro_lang/src/sim/builder.rs`; the decision itself is `FaultState::filter`). The filter delivers the message, drops it, or blocks it because its sender has crashed.

There are two kinds of fault:

- **Omission.** Drop one message. A message is identified by its channel, sender, receiver, and serialized bytes. An identical retransmission is the same message, so it is dropped too.
- **Crash.** The node stops sending at some point. Every later send is dropped, and its outputs to the outside world are suppressed. A crash is a pattern of omissions: everything the node would have sent after one point.

The program's transport declarations decide which faults are allowed:

- On a lossy channel (for example `lossy_delayed_forever`), messages can be omitted.
- On a `fail_stop` channel, messages are not omitted. They are lost only when their sender crashes.

The test decides which nodes may crash, with `.crashable(label, &process)` and `.crashable_cluster(label, &cluster, members)`. Other nodes never crash.

Naming a crash point

Hydrofrac has two ways to say where a crash happens (`CrashModel`):

- **Positional** (`CrashModel::Positional`, the default). Written `crash(source after sending bcast 2 to member 1)`. The node completes that send and nothing after it.
- **Sent set** (`CrashModel::SentSet`). Written `crash(source having sent only bcast 1 to member 0, ...)`. The node gets out exactly the listed messages. Any other send is dropped, in whatever order the node attempts it.
- **Auto** (`CrashModel::Auto`). Sent sets where they allow one interleaving per fault set (section 2.2), positional everywhere else.

Section 2.2 explains why there are two models.

When faults may happen

A test body can call `run.end_of_finite_failures()`. After that point no new fault happens. A node that already crashed stays crashed. The rest of the run is a recovery window. This is Molly's fault model, in which all failures happen before a fixed time. Without the call, faults may happen at any point in the run. A protocol that recovers by retrying needs the call, followed by timer firings that let it retry.

How many crashes

By default there is no limit on the number of crashes in one fault set. Two kinds of fault set are skipped, because they cannot show a disagreement:

- every node crashing; and
- every replica of the checked cluster crashing, or all but one. A single survivor always agrees with itself.

A test can set a limit with `.crash_budget(k)`, for a protocol with a stated tolerance such as "one crash out of three acceptors". Only multi-Paxos uses it in the corpus. The search prints its limits before the first run:

```
ldfi: crash bounds: crashes unbounded; not all 4 nodes at once; at most 1 of the checked cluster's 3
crashable member(s)
```

2.2 Interleavings

Two searches run together. Hydrofrac chooses a fault set. The simulator then explores the interleavings of message deliveries and node steps under that fault set. For each fault set, the test picks one of two modes:

- `Exploration::Exhaustive` (the default) runs every interleaving the simulator can produce. A pass is a certificate.
- `Exploration::Fuzz { instances: n }` runs `n` random interleavings. A pass is "no violation found": evidence, not a proof.

One failing interleaving is enough for a counterexample.

When one interleaving is enough

The EC label means the labelled computation does not depend on arrival order. Now suppose nothing upstream of the checked collection is non-deterministic either. In Hydro, the non-deterministic operators are the ones whose API takes a `nondet!` guard:

- `batch` and `snapshot` (and the `sample_*` operators that contain them);
- `assume_ordering` and `assume_retries`;
- `MergeOrdered`;
- timers (`source_interval`, `Instant::now`).

If none of these is upstream, and every fault names a specific message, then each node receives the same set of messages in every interleaving. So each node reaches the same final state, and one interleaving decides the fault set.

Positional crashes break this argument. `crash(n after m)` cuts off whatever `n` had not sent yet, and that depends on the order in which `n` attempts its sends. A sent-set crash does not have this problem: a send gets out if and only if it is in the set, in any order. So hydrofrac runs one interleaving per fault set (`Exploration::FirstSchedule`) when all of these hold:

- the crash model is `SentSet`, or `Auto` resolving to it;
- the check is the generated EC check;
- the static check (`hydro_lang/src/sim/order.rs`) finds no untrusted non-deterministic operator upstream of the checked collection.

The search says which case applies:

```
ldfi: one schedule per hypothesis: no non-deterministic operator upstream of the checked collection, sent-set crashes
ldfi: schedules explored (Exhaustive): order-insensitive upstream, but crash points are positional (`CrashModel::SentSet` would reduce to one schedule)
ldfi: schedules explored (Fuzz { instances: 8 }): non-deterministic upstream: {batch at quorum.rs:135 (Cluster(loc2v1))}
```

Why sent sets are not the default

Sent sets are weaker than positional crashes in two ways:

- A set is built from a run that already happened. It cannot let through a message the node first sends only under a later fault set.
- A set cannot say "the first copy of a message but not the second", because identical copies are the same message.

ISR replication shows the cost. After a replica's copy is lost, the ISR leader must shrink the in-sync set (send `Shrink([1])` to ZooKeeper) before it acknowledges. No earlier run sent that message, so no proposed set contains it. With sent sets, the unclean-election bug went from a counterexample to a false certificate. Both weaknesses come from code that reacts to faults: retries, heartbeats, view changes. That code is non-deterministic, and the static check already marks it as such. `Auto` therefore keeps positional crashes, and interleaving exploration, wherever the static check finds a non-deterministic operator upstream. On every search in the corpus, `Auto` gave the same verdict as `Positional`.

The static check is conservative. A `batch` whose consumer does not care about batch boundaries is still flagged. A test can vouch for such a site with `.trust_order_insensitive(["quorum.rs:135"])`. The certificate then names the assumption:

```
LDFI: no fault set within bounds violates the goal under any schedule (132 hypothesis run(s), one schedule
each: ASSUMING the non-deterministic operators {batch at quorum.rs:135 (Cluster(loc2v1))} order-
insensitive (trusted by the test), sent-set crashes)
```

In a `search_all_labels` report, the same row reads `certified (N fault sets, one schedule each: order-insensitive upstream, trusting batch at quorum.rs:135)`.

Which corpus labels the static check classifies as order-insensitive:

- **Order-insensitive:** the reliable-broadcast family (closed, live, cluster to cluster), the single-hop `broadcast_closed` labels, multi-Paxos `prepares` (`multi_paxos.rs:281`), ISR `writes_at_replicas` (`:343`), 2PC `prepares` (`:86`), and ABD `queries` (`:141`).
- **Not order-insensitive** (interleavings still explored): multi-Paxos `learned` and its other labels (batches at `multi_paxos.rs:299`, `:436`, `quorum.rs:135`), G-Set gossip, `leader_merge`, ISR views, uniform broadcast (`quorum.rs:135`), the `hydro_test` reliable-broadcast harness, and every hand-written specification.

Memory

The fault-free first run (which learns the facts to check, section 2.4) used to be explored like any other fault set. On uniform broadcast that meant more than 359,000 interleavings and 10 GB of memory, and it never finished. Now each interleaving is folded into the search state as soon as it ends, and its event log is dropped. The same search with trusted sent sets peaks at about 250 MB for the whole test process.

2.3 Lineage

Lineage is the record of which message deliveries an output depended on. One set of deliveries that together produced the result is a **derivation**. Hydrofrac reads derivations from passing runs and asks an integer linear programming solver (HiGHS) for the smallest fault set that removes at least one delivery from every derivation, within the crash limits. If the solver finds none, every allowed fault set leaves some derivation intact, and the result is a certificate.

Lineage must never miss a delivery that mattered. A missed delivery could make the search skip the fault that breaks the program, and the certificate would be wrong. Extra deliveries are safe but cost runs. So hydrofrac errs toward including too much.

It records lineage at two levels (`.level(..)`):

- **`Level::Execution`.** The simulator runs the program as a sequence of steps, each one run of a node's code. An output depends on every message delivered to the step that produced it, and on every earlier step at the same node, which stands in for the node's state. This needs nothing from the program and is always safe, but it is coarse.
- **`Level::Precise`** (the default). Each data element carries the ids of the deliveries it was computed from. This needs the program compiled with `.with_provenance()`, which rewrites the dataflow to carry the ids.

Some elements come out of user closures the rewrite cannot see into, and some programs are compiled without provenance. For those, hydrofrac re-runs the node with its deliveries removed one at a time, and keeps each delivery without which the element disappears. If that does not give a clear answer, it falls back to the execution level.

Both levels give the same verdicts. They differ in cost. In earlier measurements (`docs/design/ldfi-level-ablation.md`), `Precise` cut a two-phase-commit search with background traffic from 101 runs to 23. On Raft, which packs each step into one closure, `Execution` certified in 59 runs where `Precise` needed 107.

2.4 The check is generated from the label

By default the test writes no specification. `Spec::all_labels(&mut flow)` finds every EC-labelled collection, and `Ldfi::search_all_labels` runs one search per collection. For each one, the check is:

1. Run once without faults. Record every element any replica holds. These are the **facts**. They are fixed for the rest of the search.
2. In every later run, look only at replicas that did not crash.
3. At the end of the run, if some surviving replica holds a fact, every surviving replica must hold it.

Final contents at three survivors	Result
`{1, 2}`, `{1, 2}`, `{1, 2}`	passes
`{1}`, `{1}`, `{1}`	passes (item 2 was lost everywhere)
`{1, 2}`, `{1, 2}`, `{1}`	fails

If the fault-free run produces no facts, there is nothing to check and the result is **vacuous**.

What the check is, exactly

The check is the **agreement** property of reliable broadcast: if one correct node delivers a message, every correct node does. It leaves out the other properties:

- **Not uniform.** Crashed replicas do not count, even for what they held before crashing.
- **No validity.** A fact that every survivor lost passes.
- **No integrity.** Sets are compared, so duplicates are ignored, and so are elements that were not in the fault-free run.

Why the check has this shape

Hydro grants the EC label in few places:

- the receiving side of `broadcast_closed` in `hydro_lang`, and of the `fan_out` broadcasts (`broadcast_live` and the reliable broadcasts) in `hydro_std::ec_inference_demos`, both through a trusted assertion; and
- an explicit `assert_has_consistency_of(manual_proof!(..))`.

The label then survives only through operators that are deterministic given their inputs. So every labelled collection is "a broadcast was received, then processed deterministically". The question left open is whether the broadcast reached every survivor, and that is reliable-broadcast agreement.

The consensus parts of the corpus protocols (the Paxos chosen value, the 2PC decision, the ISR view, the ABD quorum) are computed at senders, in unlabelled code. Hydrofrac checks how their results are disseminated, not whether they are correct.

Reliable or uniform agreement?

Which property a protocol needs depends on whether a replica can expose a fact before it crashes:

- If a replica cannot affect anything outside itself before crashing, non-uniform agreement is enough. A crashed replica's contents are gone, and the survivors agree.
- If a replica can act on a fact before crashing (answer a client, write durable storage), the outside world may have seen a fact the survivors never get. Ruling that out needs **uniform** agreement: a fact delivered by any replica, even one that later crashed, reaches every survivor.

The generated check covers only the non-uniform case. The uniform-agreement tests (`hydro_test/src/cluster/uniform_agreement_ldfi.rs`, section 3.5.7) show the difference. Plain reliable broadcast passes the generated check but fails uniform agreement: the source sends item 2 only to member 0, member 0 delivers it, echoes it to itself, and crashes; the survivors never get it. Uniform broadcast (deliver after two echoes) passes uniform agreement in sampled interleavings. The hand-written uniform check needed a precondition, "some member delivered something". Without it, a run in which nothing was delivered passed trivially and produced a false certificate.

Checking inputs is enough

Labelled computation is deterministic given its inputs. So it is enough to check that the inputs reach every surviving replica, at the point where they are combined. Hydrofrac uses this in two ways:

- **Covered by input** (`LabelVerdict::CoveredBy`). A labelled single value (`Singleton`) built by folding an EC stream is not checked itself. It takes the verdict of the stream it folds. Example: the G-Set state `gset_state` is covered by `all_elements`.
- **Hop** (`LabelVerdict::Hop`). A labelled stream that is merged with other labelled streams before it leaves the labelled region is informational. The merge decides. Example: in multi-Paxos, the proposer's broadcast to learners (`initial`) and the learners' echo (`echo`) each diverge alone, but their union `learned` agrees.

Input agreement is sufficient, not necessary. In ABD, a replica that misses a write that a later write overrides still ends with the same register. Hydrofrac reports the missing input anyway.

Hydrofrac trusts `manual_proof!` assertions. A wrong proof is a bug in the first half of the promise, which is out of scope.

A note on `fail_stop`

`hydro_lang/src/networking/mod.rs` justifies EC for `fail_stop` by recipient failure: "A failed connection stops all future deliveries to that recipient, which models the recipient as having failed." It does not cover a sender that crashes partway through a fan-out. Read as "same inputs, same result", the label is still correct: it promises

nothing when the inputs differ, and hydrofrac is what checks whether they do. Whether the EC label is officially this conditional promise is an open question for the Hydro team (section 4).

2.5 Verdicts

Verdict (as printed)	Meaning
`DIVERGES under <faults>`	Counterexample: a fault set and interleaving under which survivors disagree at the end of the run.
`certified (N fault sets, all schedules)`	No allowed fault set breaks the check in any interleaving.
`certified (N fault sets, one schedule each: order-insensitive upstream)`	The same, using one interleaving per fault set (section 2.2).
`no violation found (N fault sets, 8 sampled schedules each)`	Only sampled interleavings were run. Not a proof.
`VACUOUS: ...`	The fault-free run held no facts. This says something about the test, not the program.
`UNDECIDED: the hypothesis budget ran out after N fault sets`	The search hit `.max_hypotheses(n)` (default 256).
`not checked (<reason>)`	Out of scope: `local inputs only`, `Singleton (not a set-valued collection)` when no labelled input covers it, `inside a tick`, or `excluded by the test` (`Spec::all_labels_except`).
`covered by <input>: <verdict>`	A labelled scalar that takes its input stream's verdict.
`diverges alone` / `agrees alone` (under "hops merged into ...")	A hop's own result. Informational; the merge's row decides.

A **fault set** (called a hypothesis in the code and in `ldfi:` log lines) is one set of faults the search tried. The count includes the fault-free run.

A certificate covers one collection, one test, and its stated limits: cluster sizes, inputs, recovery window, and crash bounds. If a protocol would converge after more retry rounds than the test gives it, hydrofrac reports a counterexample. Only the failing run shows whether the protocol or the test is at fault.

In code, `search_all_labels` returns `LabelResults`:

- `results.at("file.rs:149")` gives the rows for collections created at one source line.
- `results.counterexamples()` lists the counterexamples (hops excluded).
- `results.all_certified_or_not_checked()` checks the whole result.

2.6 Hand-written specifications

A test can check a property the label does not express, such as uniform agreement or durability of acknowledged writes. It writes the property as a small Hydro dataflow at an observer process:

- `spec.observe(stream)` and `spec.observe_cluster(stream)` send a stream to the observer.
- `spec.correct_members(&cluster, 0..n)` gives the members the current fault set did not crash.
- `spec.pre(view)` says when a run counts. `spec.post(view)` says what must then hold. Each view is an `Optional`; it holds when non-empty at the end of the run.

A run in which `pre` holds and `post` does not is a counterexample. This one says "if any correct member delivered payload 42, every correct member did" (`hydro_test/src/cluster/ldfi_ablation_broadcast.rs`):

```
let mut spec = Spec::new(&mut flow);
let delivered = spec.observe_cluster(deliveries);
let correct = spec.correct_members(&members, 0..members_n);
let delivered_m = delivered.filter(q!(|(_, p)| p.0 == 42));
spec.pre(delivered_m.clone().first());
let delivering = delivered_m.map(q!(|(m, _)| (m, ()))).unique();
let correct_delivering = correct.clone().map(q!(|m| (m, ()))).join(delivering).count();
spec.post(correct_delivering.zip(correct.count()).filter(q!(|(d, c)| d == c)));
```

Then call `Ldfi::new()...spec(&spec).search(&compiled, body)`. `spec.eventual_consistency(observed, correct)` builds the generated check on one chosen collection.

Two caveats:

- A certificate for a hand-written specification is printed as `RELATIVE`. The search assumes that a fault set leaving a derivation intact cannot break the property. That holds for the generated check by construction, but not for every property. For example, an extra conflicting fact can break agreement without removing any delivery.
- The automatic crash bound per cluster ("not all, or all but one, of the checked cluster") applies only to the generated check. A hand-written specification must encode its own bound, for example in `pre`, or use `.crash_budget(k)`, which counts crashes over all nodes.

3. How to use it

3.1 Setup

Hydrofrac is behind the `ldfi` Cargo feature of `hydro_lang`. The dev-dependencies of `hydro_test` and `hydro_std` already enable it.

The feature builds the HiGHS solver from source. That needs a C++ compiler, `cmake`, and `libclang` (for the Rust bindings). Without root access, the Python packages work:

```
python3 -m venv ~/hf-venv
~/hf-venv/bin/pip install cmake libclang
export PATH="$(echo ~/hf-venv/lib/python3*/site-packages/cmake/data/bin):$PATH"
export LIBCLANG_PATH="$(echo ~/hf-venv/lib/python3*/site-packages/clang/native)"
```

Run each test alone, with its output shown, from the repository root:

```
cargo test -p hydro_test --lib <module path>::tests::<test name> -- --nocapture --test-threads=1 --exact
```

The first test run compiles the simulated program and takes a minute or two. Later runs reuse the build.

3.2 A minimal test

This test sends two items from one process to three replicas, and lets every node crash. It is `ec_broadcast_closed_all_labels` in `hydro_test/src/cluster/ec_ldfi.rs`:

```
let mut flow = FlowBuilder::new();
let source = flow.process::<Source>();
let members = flow.cluster::<Member>();
let (send, items) = source.sim_input::<Item, TotalOrder, ExactlyOnce>();

// The transport labels this collection EventualConsistency.
let at_members = items.broadcast_closed(&members, TCP.fail_stop().name("bcast").bincode());
let _out = at_members.weaken_consistency().sim_cluster_output();

let spec = Spec::all_labels(&mut flow); // names no collection

let compiled = flow.sim()
    .skip_consistency_assertions()
    .with_cluster_size(&members, 3)
    .with_provenance()
    .compiled();

let results = Ldfi::new()
    .crashable("source", &source)
    .crashable_cluster("member", &members, 0..3)
    .spec(&spec)
    .search_all_labels(&compiled, async |_run| {
        send.send(Item(1));
        send.send(Item(2));
        hydro_lang::sim::quiesce().await;
    });
```

The test supplies the program, the cluster size, which nodes may crash, and a body that sends inputs and waits for the program to go idle (`quiesce`). It does not say what to check.

3.3 Reading the output

`search_all_labels` prints a report at the end:

```
hydrofrac: EventualConsistency results
members (3 members)
  at_members  ec_ldfi.rs:270  DIVERGES under crash(source after sending bcast 2 to member 1)

summary: 0 hold, 1 DIVERGE (members/at_members)
```

- The report has one block per cluster, named by its `crashable_cluster` label (`member` becomes `members`).
- Each row is one labelled collection, named by the variable it is bound to (`at_members`), then its source line.
- Faults name channels by their `.name(..)` (`bcast`) or by the variable the send is bound to, and members by label and id (`member 1`).
- Within a block, searched collections come first, then covered scalars, then those not checked, then hops nested under the collection they merge into. A file name is printed once per report; later rows show only `:line`.
- The summary counts the rows that hold and lists those that diverge.

Here the source crashes right after sending item 2 to member 1, so member 2 never gets item 2.

Set `HYDROFRAC_VERBOSE=1` (or print with `{:#}`) to add each row's exact identity, the fault in the internal notation, and the static order check:

```
  at_members  ec_ldfi.rs:270  DIVERGES under crash(source after sending bcast 2 to member 1)
      ec_ldfi.rs:270 (AssertIsConsistent, minted and delivered, Cluster(loc2v1)): violated by
{crash(source after bcast[→1]:Item(2))} after 5 hypothesis runs; order-insensitive upstream
```

`minted and delivered` means the label is granted at this collection and leaves the labelled region here.

Before the report, each search prints `ldfi:` lines: the facts learned, the crash bounds, how interleavings are explored, and one line per fault set tried. A counterexample also prints the deliveries of the failing run:

```
hydrofrac: checking members / at_members (ec_ldfi.rs:270)
ldfi: fact domain learned from the failure-free run: 2 fact(s)
ldfi: crash bounds: crashes unbounded; not all 4 nodes at once; at most 1 of the checked cluster's 3
crashable member(s)
ldfi: run #1 {}: 1 schedule(s), goal held; derivations {{bcast[→0]:Item(1)←source#0, ...}}
ldfi: run #2 {crash(member@0 before any send)}: 1 schedule(s), goal held; derivations {...}
ldfi: run #3 {crash(member@1 before any send)}: 1 schedule(s), goal held; derivations {...}
ldfi: run #4 {crash(member@2 before any send)}: 1 schedule(s), goal held; derivations {...}
ldfi: run #5 {crash(source after bcast[→1]:Item(2))}: 1 schedule(s), goal VIOLATED; derivations {}
LDFI: goal violated after 5 hypothesis run(s) by {crash(source after bcast[→1]:Item(2))}
  delivered in the failing schedule: {bcast[→0]:Item(1), bcast[→1]:Item(1), bcast[→2]:Item(1),
bcast[→0]:Item(2), bcast[→1]:Item(2)}
  sends lost to crashes: {source#5}
```

Runs 2 to 4 each crash one member. Each removes part of the derivation, but none can break agreement, because a crashed member does not count. Run 5 crashes the source between two sends.

A test that checks one collection with `Ldfi::search` gets an `Outcome` instead of a report. Its `Display` is the `LDFI:` line above.

3.4 Environment variables

Variable	Values	Effect
`LDFI_CRASH_MODEL`	`positional`, `sent-set`, `auto`	Crash model, unless the test sets `.crash_model(..)`. Default `positional`.
`LDFI_TRUST_ORDER_INSENSITIVE`	comma-separated sites, e.g. `quorum.rs:135`	Adds trusted sites (section 2.2).
`LDFI_EXPLORATION`	`exhaustive`, `fuzz:N`, `first`	Overrides the test's exploration. `first` runs one interleaving without claiming a reduction.
`LDFI_SCHEDULE_REDUCTION`	`0`	Turns off the one-interleaving reduction, unless the test sets `.schedule_reduction(..)`.
`HYDROFRAC_VERBOSE`	`1`	Verbose report rows.
`MOLLY_VIZ_OUT`	a directory	Where gallery tests write Lamport diagrams (section 3.6).

Variable	Values	Effect
`LDFI_RECORD`	a file path	Appends one tab-separated line per search: test, label, crash model, exploration, verdict, faults, fault sets, interleavings, seconds, peak memory (MB), exploration note.
`LDFI_LEVEL`	`exec`, `precise`	Lineage level, in tests that call `.level_from_env()`.

3.5 The examples

Each example gives the test code (trimmed to what differs from section 3.2), the command, the output, the Lamport diagram if one is committed, and the cost.

About the cost numbers. "Fault sets", "interleavings" and "time" come from `LDFI_RECORD` on the current tree. Time is the search itself, not compilation. Each test ran once, four at a time on 8 cores, so times are rough (about ±20%). Sampled searches can vary by one or two fault sets between runs. "Brute force" is `Strategy::Blind`, measured once on an earlier version of the code (section 3.7). Where the lineage count has changed since then by more than sampling noise, the older lineage count is given too.

3.5.1 One-hop broadcasts

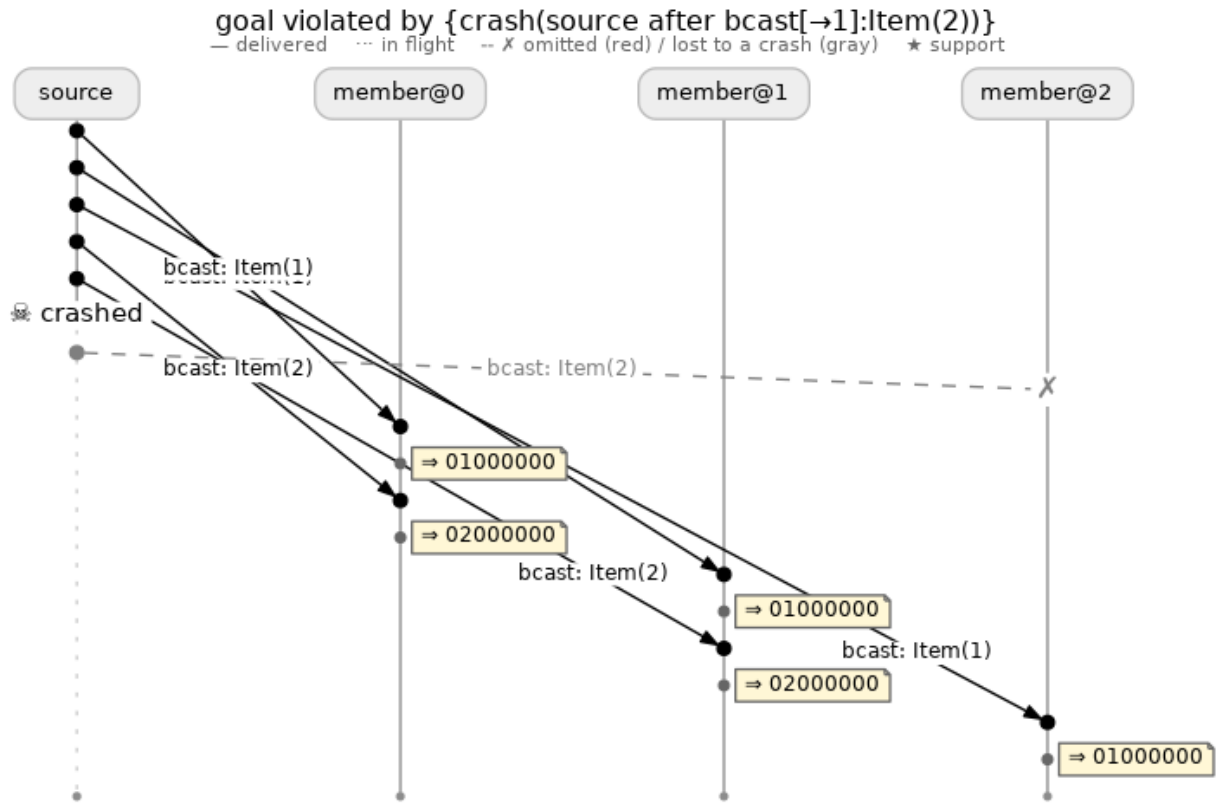
These programs send each message once and never repair a loss. All of them diverge, which is correct. They show that a transport's label alone does not make a collection eventually consistent.

`broadcast_closed`, process to cluster

The test in section 3.2. The same check can be written by hand on the one collection (`ec_broadcast_closed_fail_stop_breaks_under_a_sender_crash`), with the same result.

```
cargo test -p hydro_test --lib cluster::ec_ldfi::tests::ec_broadcast_closed_all_labels -- --nocapture --
test-threads=1 --exact
```

```
members (3 members)
  at_members  ec_ldfi.rs:270  DIVERGES under crash(source after sending bcast 2 to member 1)
```



Cost: 5 fault sets, 5 interleavings, 0.06 s. Brute force: 2. Brute force wins here: a source crash is second in its order. Hydrofrac first tries crashing each member, which removes part of the derivation but cannot break agreement.

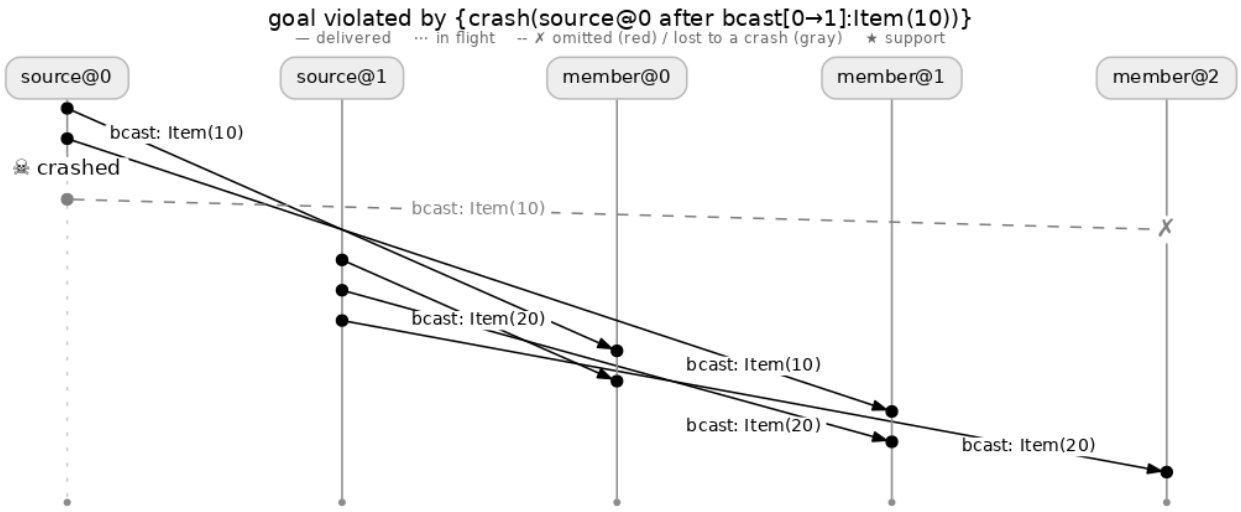
`broadcast_closed`, cluster to cluster

Two senders each broadcast one item to three members.

```
let sources = flow.cluster::<Source>();   // 2 members
let at_members = items.broadcast_closed(&members, TCP.fail_stop().name("bcast").bincode()).values();
// .crashable_cluster("source", &sources, 0..2).crashable_cluster("member", &members, 0..3)
// body: send.send(0, Item(10)); send.send(1, Item(20)); quiesce
```

```
cargo test -p hydro_test --lib
cluster::ec_ldfi::tests::ec_broadcast_closed_cluster_to_cluster_breaks_under_a_source_crash -- --nocapture
--test-threads=1 --exact
```

```
LDFI: goal violated after 5 hypothesis run(s) by {crash(source@0 after bcast[0→1]:Item(10))}
```



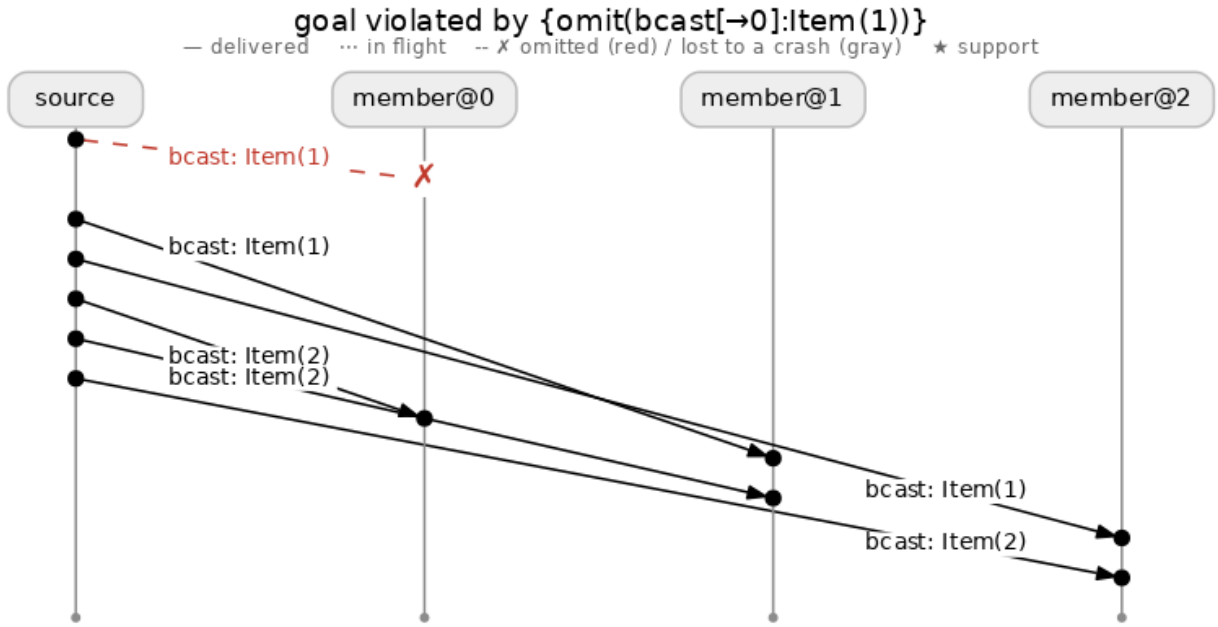
Cost: 5 fault sets, 5 interleavings, 0.04 s. Brute force: 2.

`broadcast_closed` over a lossy channel

```
let at_members = items.broadcast_closed(&members, TCP.lossy_delayed_forever().name("bcast").bincode());
```

```
cargo test -p hydro_test --lib
cluster::ec_ldfi::tests::ec_broadcast_closed_lossy_delayed_forever_breaks_under_one_omission -- --
nocapture --test-threads=1 --exact
```

```
LDFI: goal violated after 2 hypothesis run(s) by {omit(bcast[→0]:Item(1))}
```



Cost: 2 fault sets, 2 interleavings, 0.01 s. Brute force: 2.

The transport treats a lost message as "delayed forever". Nothing resends it, so member 0 never gets `Item(1)`.

`broadcast_live_from_process`

The `hydro_std` broadcast whose fan-out reads the live membership list.

```
let delivered = broadcast_live_from_process(items, &members, TCP.fail_stop().name("bcast").bincode())
    .weaken_consistency();
```

```
cargo test -p hydro_test --lib
cluster::ec_demos_ldfi::tests::ec_demo_broadcast_live_breaks_under_a_sender_crash -- --nocapture --test-
threads=1 --exact
```

```
LDFI: goal violated after 5 hypothesis run(s) by {crash(source after bcast[→1]:2)}
```

No diagram. Cost: 5 fault sets, 5 interleavings, 0.05 s. Brute force: 2.

`leader_merge_broadcast`

Two writers send to a leader, which merges the writes and broadcasts them to three replicas.

```
let at_leader = writes.send(&leader, TCP.fail_stop().name("to_leader").bincode());
let merged = leader_merge_broadcast(at_leader, &replicas, TCP.fail_stop().name("merged").bincode(), nondet!
(..));
// .crashable("leader", &leader).crashable_cluster("replica", &replicas, 0..3)
```

```
cargo test -p hydro_test --lib
cluster::ec_demos_ldfi::tests::ec_demo_leader_merge_breaks_under_a_leader_crash -- --nocapture --test-
threads=1 --exact
```

```
LDFI: goal violated after 2 hypothesis run(s) by {crash(leader after merged[→1]:(MemberId::<...Writer>(0),
100))}
```

No diagram. Cost: 2 fault sets, 4 interleavings, 0.04 s. Brute force: 4.

The program is compiled without `.with_provenance()`, because the rewrite does not support `entries_partially_ordered`. The search replays nodes instead (section 2.3). The demo's own documentation lists leader survival as an assumption. Hydrofrac shows what happens when it fails.

A label inside a tick

A tick is one step of a node's local computation. A collection inside a tick holds only that step's data. Here a broadcast is batched into each member's tick, labelled there by hand, and flattened back out.

```
let at_members = items.broadcast_closed(&members, TCP.fail_stop().name("bcast").bincode());
let in_tick = at_members.batch(&tick, nondet!(..))
    .assert_has_consistency_of::<Tick<Cluster<'_, Member, EventualConsistency>>>(manual_proof!(..))
    .map(q!(|x| x * 10));
let out = in_tick.all_ticks().weaken_consistency().sim_cluster_output();
```

```
cargo test -p hydro_test --lib
cluster::ec_all_labels_classes::tests::all_labels_reports_a_label_inside_a_tick -- --nocapture --test-
threads=1 --exact
```

```
members (3 members)
  at_members  ec_all_labels_classes.rs:31  DIVERGES under crash(source after sending bcast 2 to member 1)
  out         :41                          DIVERGES under crash(source after sending bcast 2 to member 1)
  in_tick     :36                          not checked (inside a tick)

summary: 0 hold, 2 DIVERGE (members/at_members, members/out), 1 not checked
```

No diagram. Cost: 5 fault sets and 948 interleavings for each searched row, about 4.8 s each. Brute force: 2 each. The batching multiplies the interleavings (about 190 per fault set). The row inside the tick is skipped because its contents are per tick, not final.

3.5.2 Broadcasts with redundancy

These protocols add an echo or gossip, so a message lost on one path can arrive on another. Their delivered sets agree. Lineage pays off most here, because a certificate must show that every allowed fault set leaves some path intact.

Plain and echo broadcast, side by side

Two versions of the broadcast in `hydro_test/src/cluster/reliable_broadcast.rs`. In one, the sender sends once. In the other, each member re-sends what it receives. Both are checked by hand on the delivered set.

```
let deliveries = broadcast(&members, source, echo, nondet!(..), nondet!(..)); // echo: bool
// .crashable("broadcaster", &broadcaster).crashable_cluster("member", &members, 0..3)
// body: source_send.send(Payload(42)); quiesce
```

```
cargo test -p hydro_test --lib
cluster::ec_ldfi::tests::ec_generated_spec_separates_plain_from_reliable_broadcast -- --nocapture --test-
threads=1 --exact
```

```
LDFI: goal violated after 2 hypothesis run(s) by {crash(broadcaster after initial[→1]:Payload(42))}
LDFI: no fault set within bounds violates the goal under any schedule (21 hypothesis run(s))
```

No diagram for this check. (`molly-viz-samples/plain_broadcast/` draws the plain version under a hand-written check.)

Version	Hydrofrac	Brute force
plain	2 fault sets, 8 interleavings, 0.03 s	2
echo (certified)	21 fault sets, 4,076 interleavings, 12.8 s	31 fault sets, 3,511 interleavings, 11.0 s

This is the one search where hydrofrac needed fewer fault sets but more time: it explored more interleavings. The harness has a `batch` upstream (`reliable_broadcast.rs:70`), so interleavings are explored.

`reliable_broadcast_closed`, every label

The `hydro_std` reliable broadcast. The source broadcasts (`initial`). Every member echoes what it receives to every member (`echo`). Each member delivers the union, deduplicated (`new_messages`).

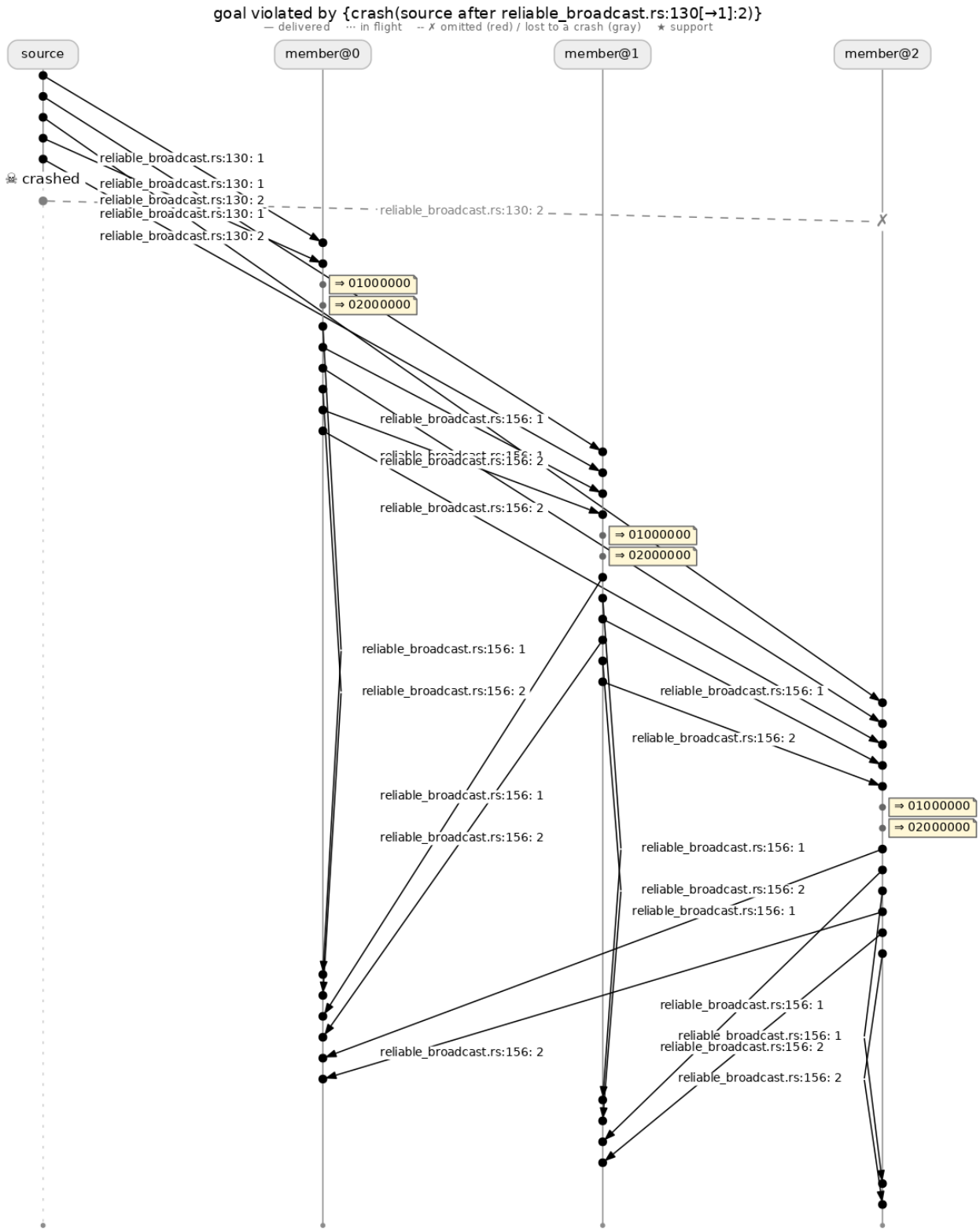
```
let delivered = reliable_broadcast_closed(items, &members);
let _out = delivered.weaken_consistency().sim_cluster_output();
let spec = Spec::all_labels(&mut flow);
// .crashable("source", &source).crashable_cluster("member", &members, 0..3)
// body: send.send(1); send.send(2); quiesce
```

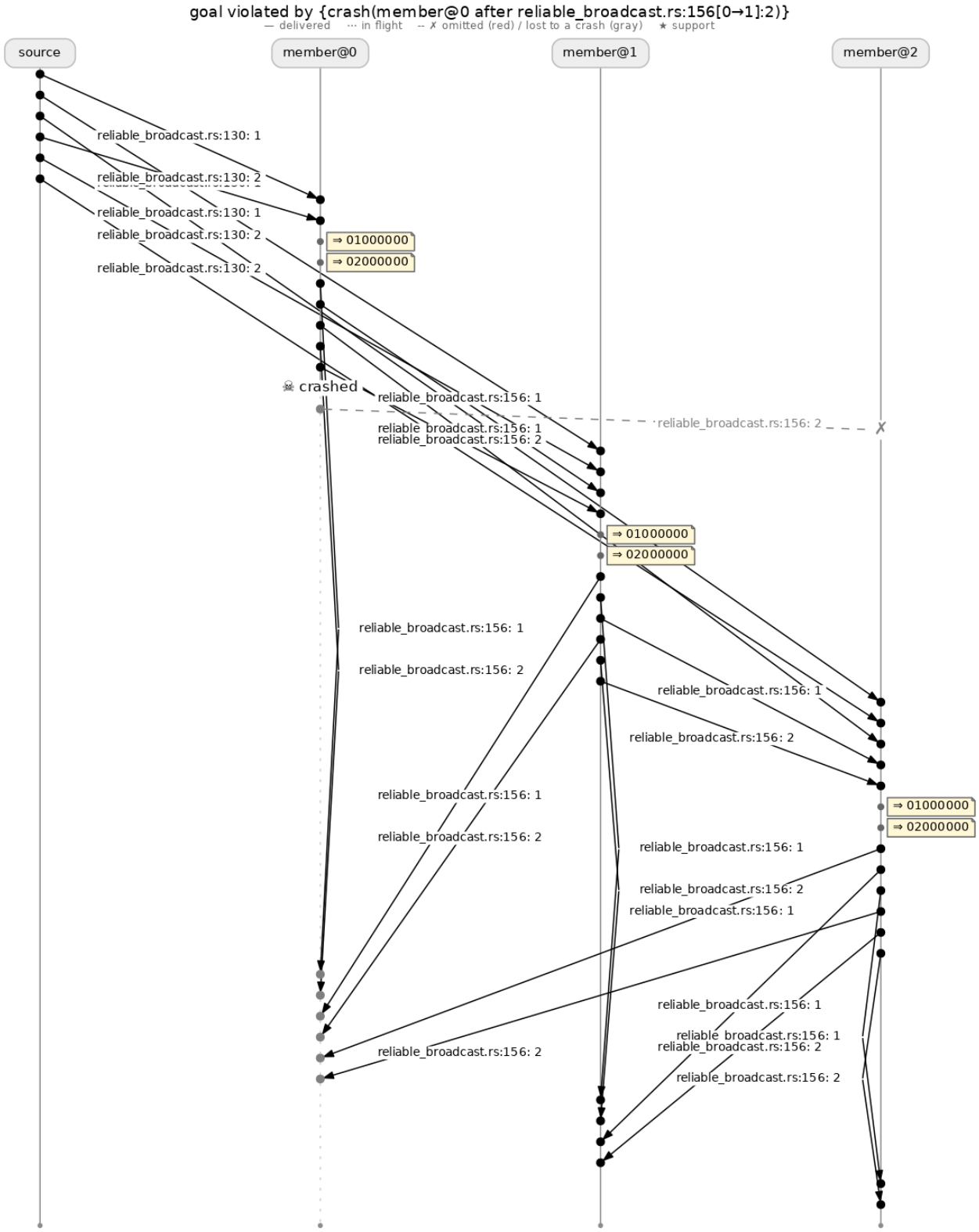
```
cargo test -p hydro_test --lib cluster::ec_demos_ldfi::tests::ec_demo_reliable_broadcast_all_labels -- --
nocapture --test-threads=1 --exact
```

```
members (3 members)
  new_messages  reliable_broadcast.rs:149  certified (49 fault sets, all schedules)
  hops merged into new_messages:
    initial  :130  diverges alone under crash(source after sending initial 2 to member 1)
    echo     :156  diverges alone under crash(member@0 after sending echo 2 to member 1)

summary: 1 hold
```

The two hops are one-hop broadcasts, so each diverges alone. Their union is the delivered set, which agrees: the echo repairs what a source crash leaves out, and the source's broadcast repairs what an echoing member's crash leaves out.


goal violated by {crash(source after reliable_broadcast.rs:130[→1]:2)}
— delivered ··· in flight -- ✗ omitted (red) / lost to a crash (gray) ★ support
source
member@0
member@1
member@2
crashed
reliable_broadcast.rs:130: 1
reliable_broadcast.rs:130: 1
reliable_broadcast.rs:130: 2
reliable_broadcast.rs:130: 1
reliable_broadcast.rs:130: 2
reliable_broadcast.rs:130: 2
⇒ 01000000
⇒ 02000000
reliable_broadcast.rs:156: 1
reliable_broadcast.rs:156: 2
reliable_broadcast.rs:156: 2
⇒ 01000000
⇒ 02000000
reliable_broadcast.rs:156: 1
reliable_broadcast.rs:156: 2
reliable_broadcast.rs:156: 1
reliable_broadcast.rs:156: 2
⇒ 01000000
⇒ 02000000
reliable_broadcast.rs:156: 1
reliable_broadcast.rs:156: 2
reliable_broadcast.rs:156: 1
reliable_broadcast.rs:156: 1
reliable_broadcast.rs:156: 2
reliable_broadcast.rs:156: 2
reliable_broadcast.rs:156: 2



Row	Hydrofrac	Brute force
`new_messages` (certified)	49 fault sets, 49 interleavings, 0.63 s	115 (lineage then took 36)
`initial` hop	5, 0.05 s	20

Row	Hydrofrac	Brute force
`echo` hop	3, 0.04 s	2

The certificate needs one interleaving per fault set even under positional crashes here. With `LDFI_CRASH_MODEL=auto`, it becomes `certified (77 fault sets, one schedule each: order-insensitive upstream)` in 1.2 s. Sent sets need more fault sets (section 2.2), but each one is decided by a single interleaving whatever the program's emission order.

Reliable broadcast, closed and live, checked by hand

The delivered sets of `reliable_broadcast_closed` and `reliable_broadcast_live` (echo over live membership), each checked by hand.

```
cargo test -p hydro_test --lib cluster::ec_demos_ldfi::tests::ec_demo_reliable_broadcast_certifies -- --
nocapture --test-threads=1 --exact
```

```
LDFI: no fault set within bounds violates the goal under any schedule (49 hypothesis run(s))
LDFI: no fault set within bounds violates the goal under any schedule (36 hypothesis run(s))
```

No diagram (nothing fails). Cost: closed 49 fault sets in 0.59 s, live 36 in 0.41 s. Brute force: 115 each (lineage then took 36 for both).

The two counts differ because the closed fan-out now emits its sends in a different order (`flat_map` instead of `cross_product`). Positional crash points depend on that order (section 2.2).

`ec_demo_reliable_broadcast_certifies_in_one_schedule_per_hypothesis` runs the closed version with `CrashModel::Auto`: `certified` in 77 fault sets and 77 interleavings, 1.5 s.

Uniform reliable broadcast

`uniform_reliable_broadcast_closed` delivers an item only after a threshold number of members have echoed it. Tested with thresholds 1 and 2.

```
let delivered = uniform_reliable_broadcast_closed(items, &members, threshold).weaken_consistency();
// hand-checked with spec.eventual_consistency; Exploration::Fuzz { instances: 8 }
```

```
cargo test -p hydro_test --lib cluster::ec_demos_ldfi::tests::ec_demo_uniform_broadcast -- --nocapture --
test-threads=1 --exact
```

```
uniform(1): LDFI: no fault set within bounds violated the goal in 8 sampled schedule(s) per hypothesis
(105 hypothesis run(s)); not a proof
uniform(2): LDFI: no fault set within bounds violated the goal in 8 sampled schedule(s) per hypothesis
(105 hypothesis run(s)); not a proof
```

The only non-deterministic operator upstream is the quorum's `batch` at `quorum.rs:135`, whose `nondet!` comment argues it does not depend on batch boundaries. Trusting it gives a certificate:

```
LDFI_CRASH_MODEL=auto LDFI_TRUST_ORDER_INSENSITIVE=quorum.rs:135 \
  cargo test -p hydro_test --lib cluster::ec_demos_ldfi::tests::ec_demo_uniform_broadcast -- --nocapture --
test-threads=1 --exact
```

```
uniform(1): LDFI: no fault set within bounds violates the goal under any schedule (132 hypothesis run(s),
one schedule each: ASSUMING the non-deterministic operators {batch at quorum.rs:135 (Cluster(loc2v1))}
order-insensitive (trusted by the test), sent-set crashes)
uniform(2): LDFI: no fault set within bounds violates the goal under any schedule (132 hypothesis run(s),
one schedule each: ASSUMING the non-deterministic operators {batch at quorum.rs:135 (Cluster(loc2v1))}
order-insensitive (trusted by the test), sent-set crashes)
```

No diagram.

Mode	Hydrofrac	Brute force
default (sampled)	105 fault sets, 840 interleavings, about 5.8 s each	115 fault sets, 920 interleavings
`auto` + trusted `batch` (certified)	132 fault sets, 132 interleavings, 4.5 s each, peak 251 MB	—
exhaustive, positional	did not finish: more than 359,000 interleavings in the fault-free run alone	—

This is the closest case between lineage and brute force. Nearly every delivery feeds the result, so lineage rules out little.

G-Set gossip, every label

Three members each add one element to a grow-only set, then run three rounds of gossip. Failures end after the updates.

```
let _set = g_set_gossip(&members, updates, ticks); // or g_set_gossip_live
let spec = Spec::all_labels(&mut flow);
// .crashable_cluster("member", &members, 0..3); Fuzz { instances: 8 }
// body: each member adds 10 + m; quiesce; end_of_finite_failures();
//       then fire every member's gossip timer 3 times, quiescing after each
```

```
cargo test -p hydro_test --lib cluster::ec_demos_ldfi::tests::ec_demo_g_set_gossip_all_labels -- --
nocapture --test-threads=1 --exact
```

```
members (3 members)
  all_elements  crdt_gossip.rs:54  no violation found (13 fault sets, 8 sampled schedules each)
  gset_state    :57                covered by all_elements: no violation found
  hops merged into all_elements:
    initial  :37  diverges alone under crash(member@0 after sending initial 10 to member 1)
    echo     :84  agrees alone

summary: 2 hold
```

The live version gives the same shape, with the hop named `fan_out` (`fan_out.rs:235`).

The G-Set state `gset_state` is a `Singleton`. It is folded from `all_elements`, the merge of each member's fresh elements with the gossiped state, so it takes that stream's verdict. The first broadcast of each new element (`initial`) diverges alone when its sender crashes; gossip repairs it.

No diagram.

Search	Hydrofrac	Brute force
`all_elements`, fixed membership	13 fault sets, 104 interleavings, 3.6 s	not measured
`all_elements`, live membership	13, 104, 4.6 s	not measured
hand-checked state, fixed membership (`ec_demo_g_set_gossip_certifies`)	13, 104, 5.5 s	40
hand-checked state, live membership	13, 104, 8.4 s	40

The hand-checked test observes each member's set through a snapshot instead of using `Spec::all_labels`.

3.5.3 Multi-Paxos

`multi_paxos` from `hydro_std`, with 1 proposer, 3 acceptors and 3 learners, all crashable. The body tells the proposer to lead, waits, sends two commands, and waits again. Both tests use `.crash_budget(1)`, matching the protocol's tolerance of one failure. They sample 8 interleavings per fault set.

```
let outputs = multi_paxos(&acceptors, &learners, 2, leads, commands);
let _learned = outputs.learned.weaken_consistency().sim_cluster_output();
let spec = Spec::all_labels(&mut flow);
Ldfi::new()
    .exploration(Exploration::Fuzz { instances: 8 })
    .crashable_cluster("proposer", &proposers, 0..1)
    .crashable_cluster("acceptor", &acceptors, 0..3)
    .crashable_cluster("learner", &learners, 0..3)
    .crash_budget(1)
    .spec(&spec)
    .search_all_labels(&compiled, async |_run| {
        lead.send(0, 1);
        hydro_lang::sim::quiesce().await;
        command.send(0, 7);
        command.send(0, 8);
        hydro_lang::sim::quiesce().await;
    })
```

```
cargo test -p hydro_test --lib cluster::ec_demos_ldfi::tests::ec_demo_multi_paxos_all_labels -- --
nocapture --test-threads=1 --exact
```

```
hydrofrac: EventualConsistency results
proposers (1 member)
  learned_at_proposers  multi_paxos.rs:569  no violation found (7 fault sets, 8 sampled schedules each)
  learned_at_proposers  :570                no violation found (6 fault sets, 8 sampled schedules each)

acceptors (3 members)
  prepares     :281  DIVERGES under crash(proposer@0 after sending prepares Ts{round: 1, writer: 0} to
acceptor 1)
  accepts_fwd  :286  DIVERGES under crash(proposer@0 after sending accepts (Ts{round: 1, writer: 0}, 0, 1,
```

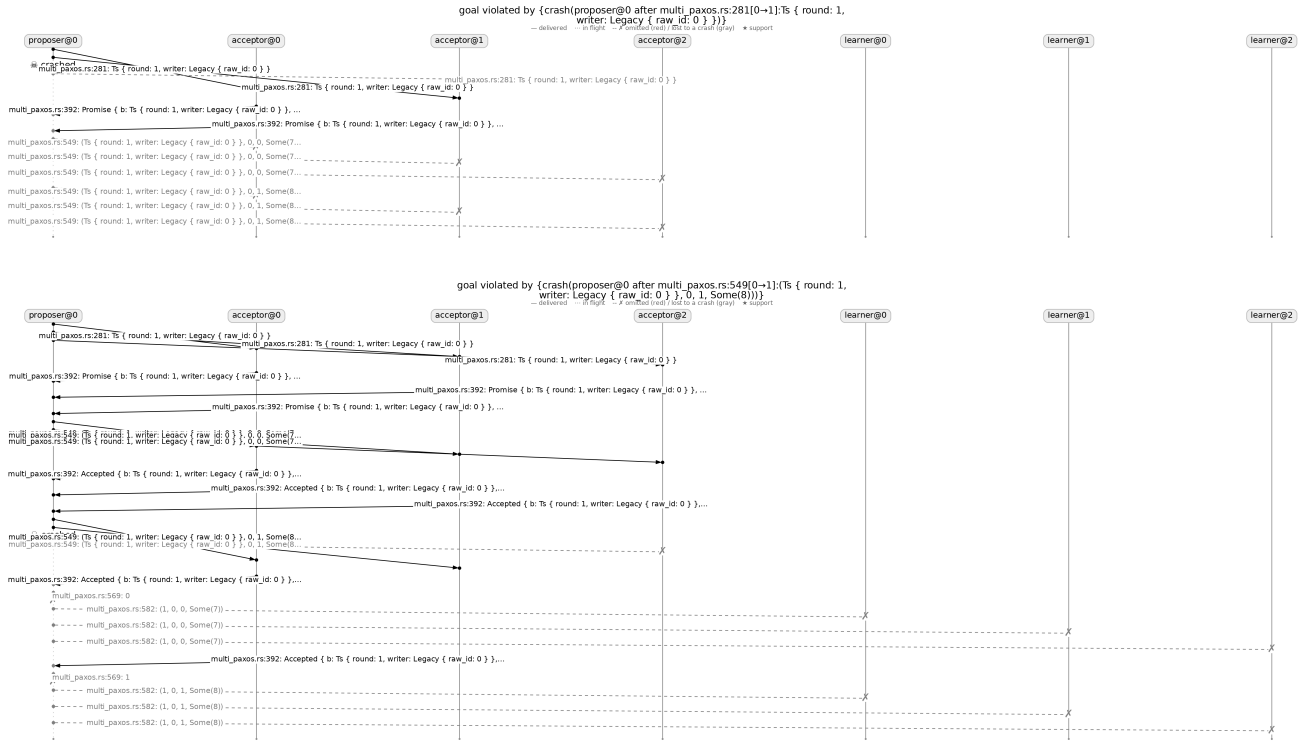
```
8) to acceptor 1)
  accepts      :549  DIVERGES under crash(proposer@0 after sending accepts (Ts{round: 1, writer: 0}, 0, 1,
8) to acceptor 1)

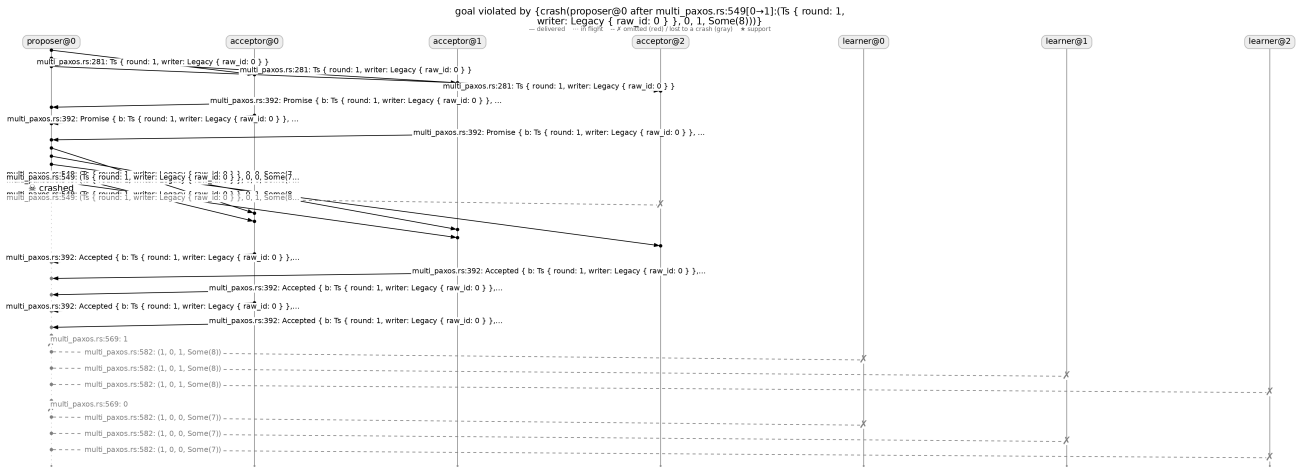
learners (3 members)
  learned           :588                no violation found (25 fault sets, 8 sampled schedules each)
  facts             :608                no violation found (34 fault sets, 8 sampled schedules each)
  splice_epoch_log  epoch_splice.rs:190  covered by facts: no violation found
  hops merged into learned:
    initial  :582  diverges alone under crash(proposer@0 after sending initial (1, 0, 0, 7) to learner 1)
    echo     :591  diverges alone under crash(learner@0 after sending echo (1, 0, 0, 7) to learner 1)

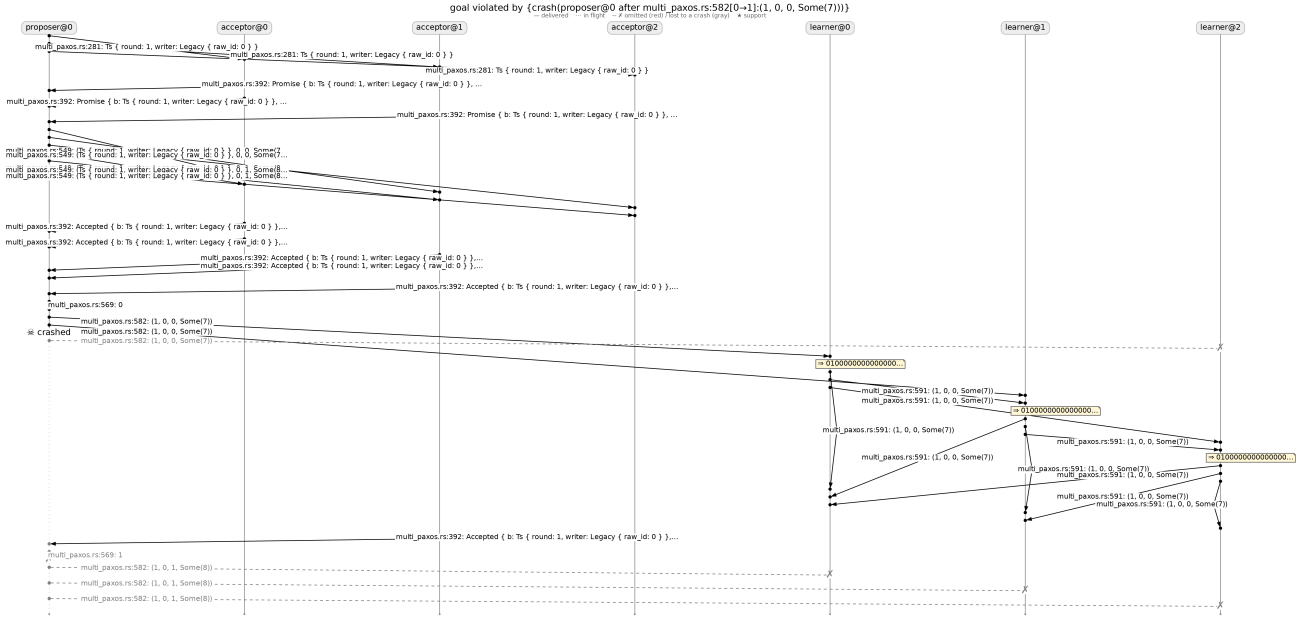
summary: 5 hold, 3 DIVERGE (acceptors/prepares, acceptors/accepts_fwd, acceptors/accepts)
```

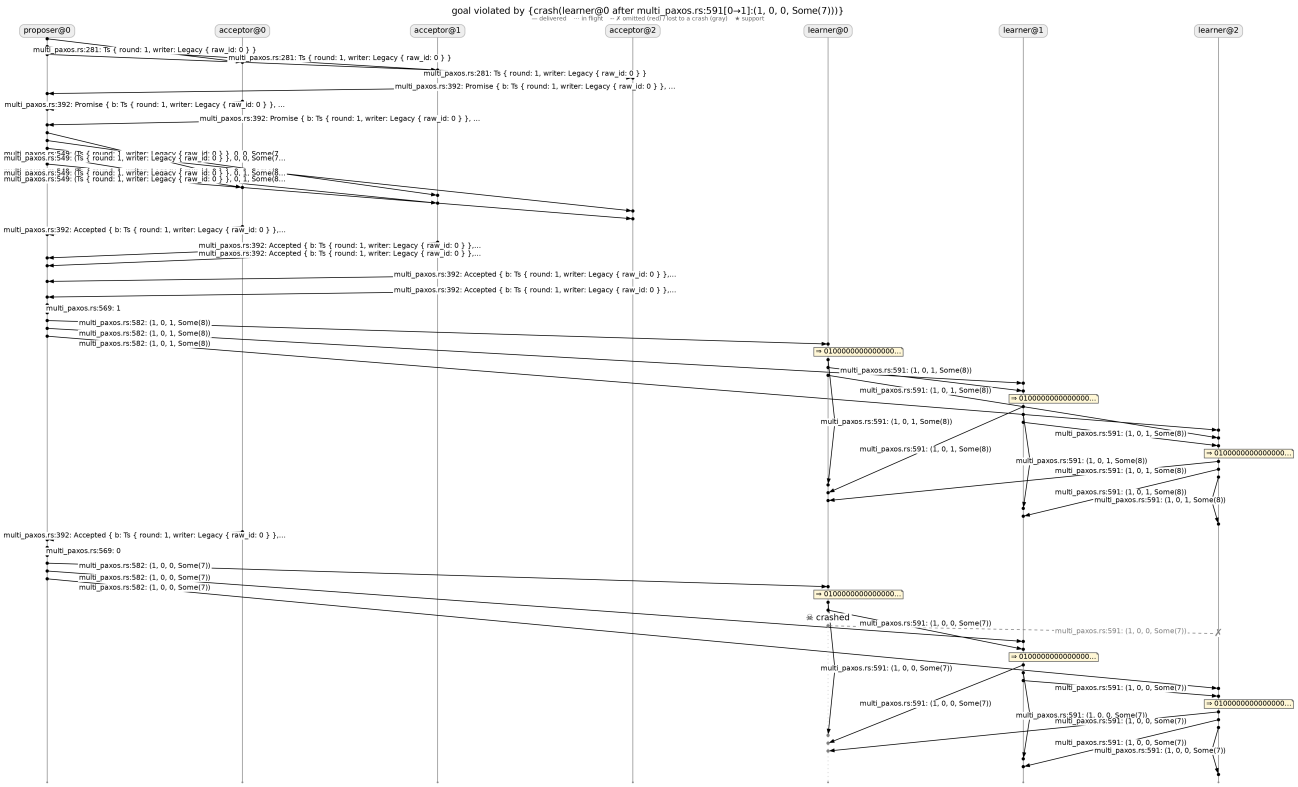
How to read it:

- `learned`, the learners' delivered set, agrees. Each learner that receives a decree echoes it to the others, so the proposer's broadcast (`initial`) and the echo (`echo`) each diverge alone, but their union does not.
- `facts` and the spliced log `splice_epoch_log` (a `Singleton` folded from `facts`) agree.
- The three acceptor rows are one-hop broadcasts from the proposer. A proposer crash partway through leaves acceptors with different prepares or accepts. Paxos does not need acceptors to agree on these. A new proposer would re-run the protocol. But the labels promise agreement, and hydrofrac reports that the promise does not hold under a proposer crash.
- The proposer rows trivially agree: there is one proposer.




goal violated by {crash(proposer@0 after multi_paxos.rs:549[0→1]:(Ts { round: 1, writer: Legacy { raw_id: 0 } }, 0, 1, Some(8)))}
— delivered · ⋯ in flight · ✗ omitted (red) / lost to a crash (gray) · ✦ support
proposer@0
acceptor@0
acceptor@1
acceptor@2
learner@0
learner@1
learner@2
multi_paxos.rs:281: Ts { round: 1, writer: Legacy { raw_id: 0 } }
multi_paxos.rs:392: Promise { b: Ts { round: 1, writer: Legacy { raw_id: 0 } }, ...
multi_paxos.rs:549: (Ts { round: 1, writer: Legacy { raw_id: 0 } }, 0, 0, Some(7...
multi_paxos.rs:549: (Ts { round: 1, writer: Legacy { raw_id: 0 } }, 0, 1, Some(8...
☠ crashed
multi_paxos.rs:392: Accepted { b: Ts { round: 1, writer: Legacy { raw_id: 0 } },...
multi_paxos.rs:569: 1
multi_paxos.rs:582: (1, 0, 1, Some(8))
multi_paxos.rs:569: 0
multi_paxos.rs:582: (1, 0, 0, Some(7))


goal violated by {crash(proposer@0 after multi_paxos.rs:582[0→1]:(1, 0, 0, Some(7)))}
— delivered · ⋯ in flight · ✗ omitted (red) / lost to a crash (gray) · ✦ support
proposer@0
acceptor@0
acceptor@1
acceptor@2
learner@0
learner@1
learner@2
multi_paxos.rs:281: Ts { round: 1, writer: Legacy { raw_id: 0 } }
multi_paxos.rs:392: Promise { b: Ts { round: 1, writer: Legacy { raw_id: 0 } }, ...
multi_paxos.rs:549: (Ts { round: 1, writer: Legacy { raw_id: 0 } }, 0, 0, Some(7...
multi_paxos.rs:549: (Ts { round: 1, writer: Legacy { raw_id: 0 } }, 0, 1, Some(8...
multi_paxos.rs:392: Accepted { b: Ts { round: 1, writer: Legacy { raw_id: 0 } },...
multi_paxos.rs:569: 0
multi_paxos.rs:582: (1, 0, 0, Some(7))
☠ crashed
⇒ 0100000000000000...
multi_paxos.rs:591: (1, 0, 0, Some(7))
multi_paxos.rs:569: 1
multi_paxos.rs:582: (1, 0, 1, Some(8))



These diagrams are wide. Versions with larger text are next to each file as `failing-print.png`.

Row	**Hydrofrac**	**Brute force**
`prepares`	5 fault sets, 40 interleavings, 0.8 s	50
`accepts_fwd`	8, 64, 1.5 s	44
`accepts`	8, 64, 1.1 s	44
`learned_at_proposers` (`:569`)	7, 56, 1.0 s	34
`learned_at_proposers` (`:570`)	6, 48, 0.9 s	34
`initial` hop	10, 80, 1.7 s	37
`echo` hop	9, 72, 1.8 s	16
`learned`	25, 200, 3.4 s	52
`facts`	34, 272, 5.6 s	52 (lineage then took 26)
total	**112 fault sets, 18 s**	**363** (lineage then took 105)

`ec_demo_multi_paxos_learned_under_one_crash` checks `learned` alone by hand: 26 fault sets, 208 interleavings, 2.6 s.

Without `.crash_budget(1)` the search did not finish in earlier measurements. It used up the default budget of 256 fault sets after 212 s. With the budget raised to 2,000, it had no verdict after 659 fault sets and 30 minutes.

3.5.4 ISR replication

A Kafka-style in-sync-replica protocol (`hydro_test/src/cluster/isr_replication.rs`): 3 replicas, a client, and a ZooKeeper stand-in (`zk`). Replication is lossy; the other channels are `fail_stop`. The body sends one write, then runs four rounds of replica heartbeats and ZooKeeper view updates. Failures end after the first round:

```
async |run| {
    h.write_send.send(Write(1));
    hydro_lang::sim::quiesce().await;
    for round in 0..4 {
        h.replica_timer.send_many((0..3).map(|m| (m, ())));
        hydro_lang::sim::quiesce().await;
        h.zk_timer.send(());
        hydro_lang::sim::quiesce().await;
        if round == 0 {
            run.end_of_finite_failures();
        }
    }
}
```

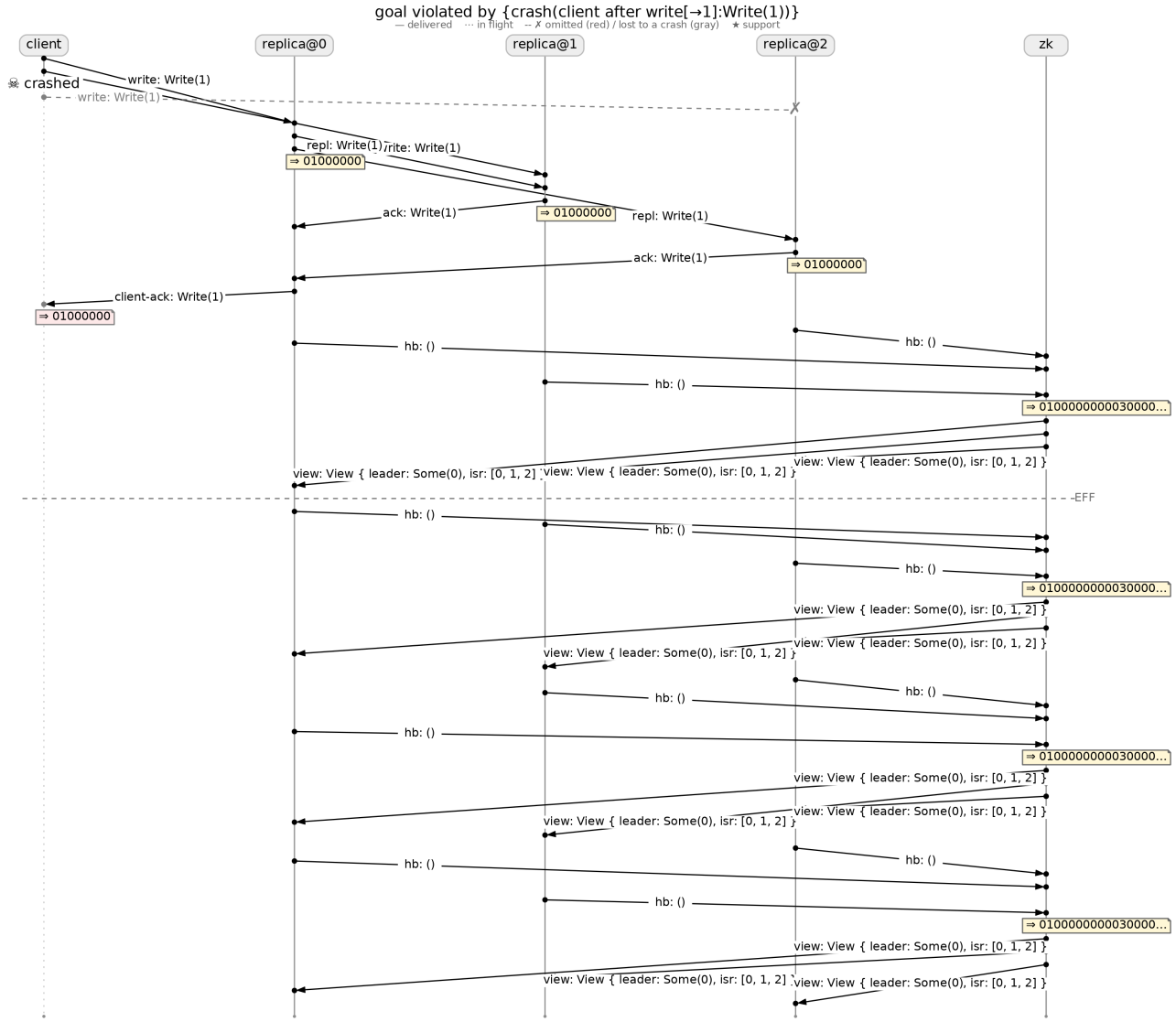
Both all-labels tests sample 8 interleavings per fault set. Three collections are labelled: the client's write broadcast (`writes_at_replicas`), and ZooKeeper's view broadcast as received (`views`) and as consumed (`views_in`).

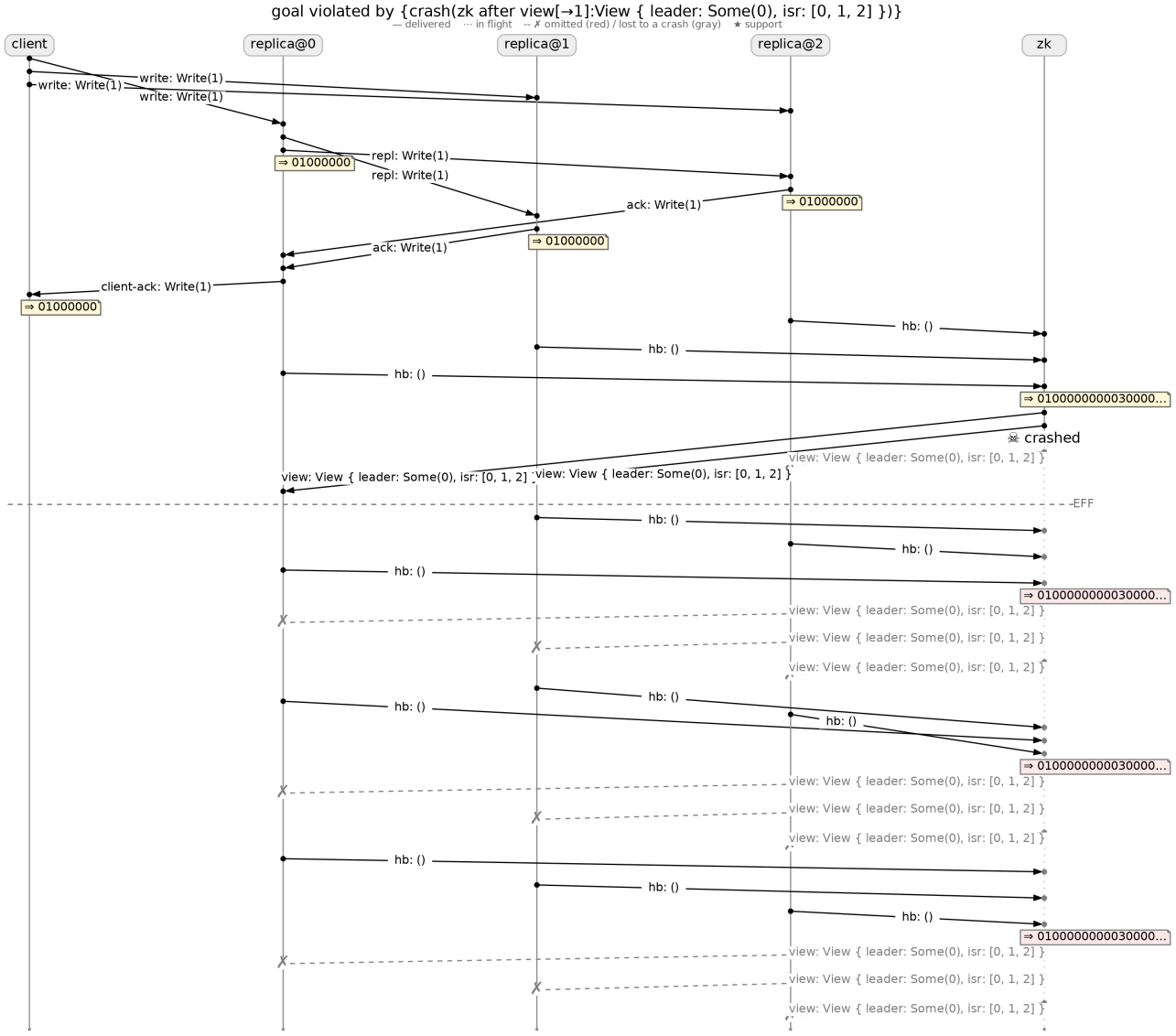
Client and ZooKeeper may crash

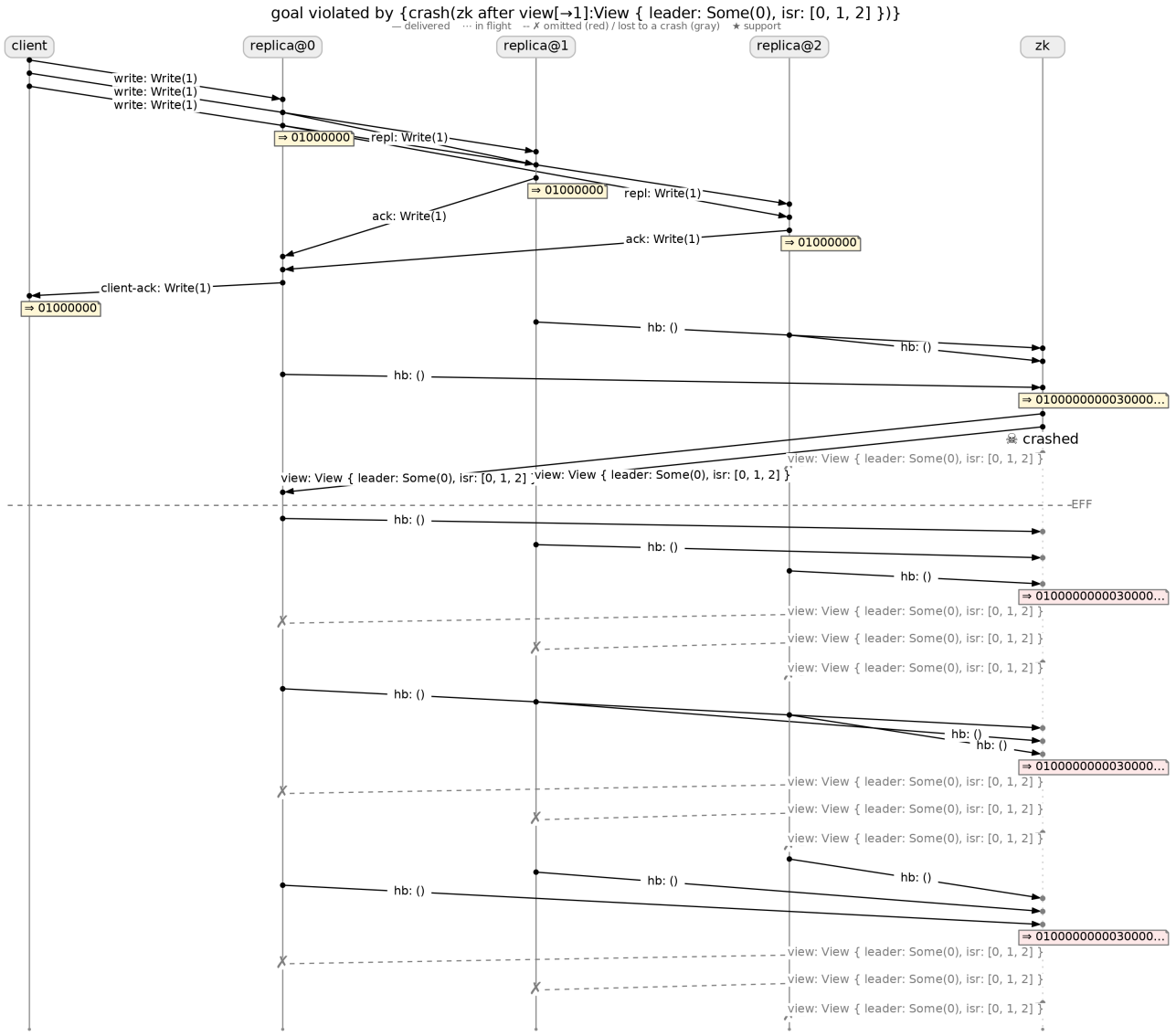
```
cargo test -p hydro_test --lib cluster::isr_replication::tests::ldfi_all_labels_break_under_a_sender_crash
-- --nocapture --test-threads=1 --exact
```

```
replicas (3 members)
  views_in            isr_replication.rs:360  DIVERGES under crash(zk after sending view View{leader: 0,
isr: [0, 1, 2]} to replica 1)
  writes_at_replicas  :343                    DIVERGES under crash(client after sending write 1 to replica
1)
  views               :564                    DIVERGES under crash(zk after sending view View{leader: 0,
isr: [0, 1, 2]} to replica 1)

summary: 0 hold, 3 DIVERGE (replicas/views_in, replicas/writes_at_replicas, replicas/views)
```


goal violated by {crash(client after write[→1]:Write(1))}
— delivered ··· in flight –– ✗ omitted (red) / lost to a crash (gray) ★ support
client
replica@0
replica@1
replica@2
zk
crashed
write: Write(1)
write: Write(1)
repl: Write(1)
rite: Write(1)
⇒ 01000000
ack: Write(1)
⇒ 01000000
repl: Write(1)
ack: Write(1)
⇒ 01000000
client-ack: Write(1)
⇒ 01000000
hb: ()
hb: ()
hb: ()
⇒ 0100000000030000…
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
EFF
hb: ()
hb: ()
hb: ()
⇒ 0100000000030000…
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
hb: ()
hb: ()
hb: ()
⇒ 0100000000030000…
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
hb: ()
hb: ()
hb: ()
⇒ 0100000000030000…
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }


goal violated by {crash(zk after view[→1]:View { leader: Some(0), isr: [0, 1, 2] })}
— delivered ⋯ in flight -- ✗ omitted (red) / lost to a crash (gray) ★ support
client
replica@0
replica@1
replica@2
zk
write: Write(1)
write: Write(1)
write: Write(1)
⇒ 01000000
repl: Write(1)
repl: Write(1)
⇒ 01000000
ack: Write(1)
⇒ 01000000
ack: Write(1)
client-ack: Write(1)
⇒ 01000000
hb: ()
hb: ()
hb: ()
⇒ 0100000000030000…
☠ crashed
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
EFF
hb: ()
hb: ()
hb: ()
⇒ 0100000000030000…
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
hb: ()
hb: ()
hb: ()
⇒ 0100000000030000…
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
hb: ()
hb: ()
hb: ()
⇒ 0100000000030000…
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }
view: View { leader: Some(0), isr: [0, 1, 2] }



Row	Hydrofrac	Brute force
`views_in`	8 fault sets, 64 interleavings, 2.0 s	24
`writes_at_replicas`	2, 16, 0.7 s	4
`views`	8, 64, 1.9 s	24

All three are one-hop broadcasts from a single process. Their only assumption is that the sender survives.

Only replicas may crash

The same test with the client and ZooKeeper held correct, as in the protocol's own fault model.

```
cargo test -p hydro_test --lib
cluster::isr_replication::tests::ldfi_all_labels_with_the_harness_fault_model -- --nocapture --test-
threads=1 --exact
```

```
replicas (3 members)
  views_in           isr_replication.rs:360  no violation found (7 fault sets, 8 sampled schedules each)
  writes_at_replicas  :343                   no violation found (4 fault sets, 8 sampled schedules each)
  views              :564                    no violation found (7 fault sets, 8 sampled schedules each)

summary: 3 hold
```

No diagram.

Row	Hydrofrac	Brute force
`views_in`	7 fault sets, 56 interleavings, 1.6 s	16
`writes_at_replicas`	4, 32, 1.0 s	68
`views`	7, 56, 1.9 s	16

`writes_at_replicas` shows the largest gap in the corpus: 4 fault sets against 68.

The unclean-election bug (hand-written specification)

This test checks that an acknowledged write survives, with a hand-written specification. It is the test that sent-set crashes get wrong (section 2.2).

```
cargo test -p hydro_test --lib cluster::isr_replication::tests::ldfi_finds_the_unclean_election_bug -- --
nocapture --test-threads=1 --exact
```

```
LDFI: goal violated after 9 hypothesis run(s) by {omit(repl[0→1]:Write(1)), crash(replica@0 after client-
ack[0→]:Write(1))}
```

The leader's replication to replica 1 is lost, the leader acknowledges the write and crashes, and an out-of-sync replica is elected. A diagram of this bug from an earlier run is in [`molly-viz-samples/isr_unclean_election/`](molly-viz-samples/isr_unclean_election/).

Cost: 9 fault sets, 72 interleavings, 28.9 s.

3.5.5 Two-phase commit

Two-phase commit with cooperative termination (`hydro_test/src/cluster/two_pc_ldfi.rs`): a coordinator, 2 participants, one transaction, every interleaving explored. With two participants, the crash bound allows no participant to crash, because one survivor always agrees with itself. Four collections are labelled: `prepares`, `commits`, the peer queries as consumed (`peer_in`), and the peer broadcast (`peer`).

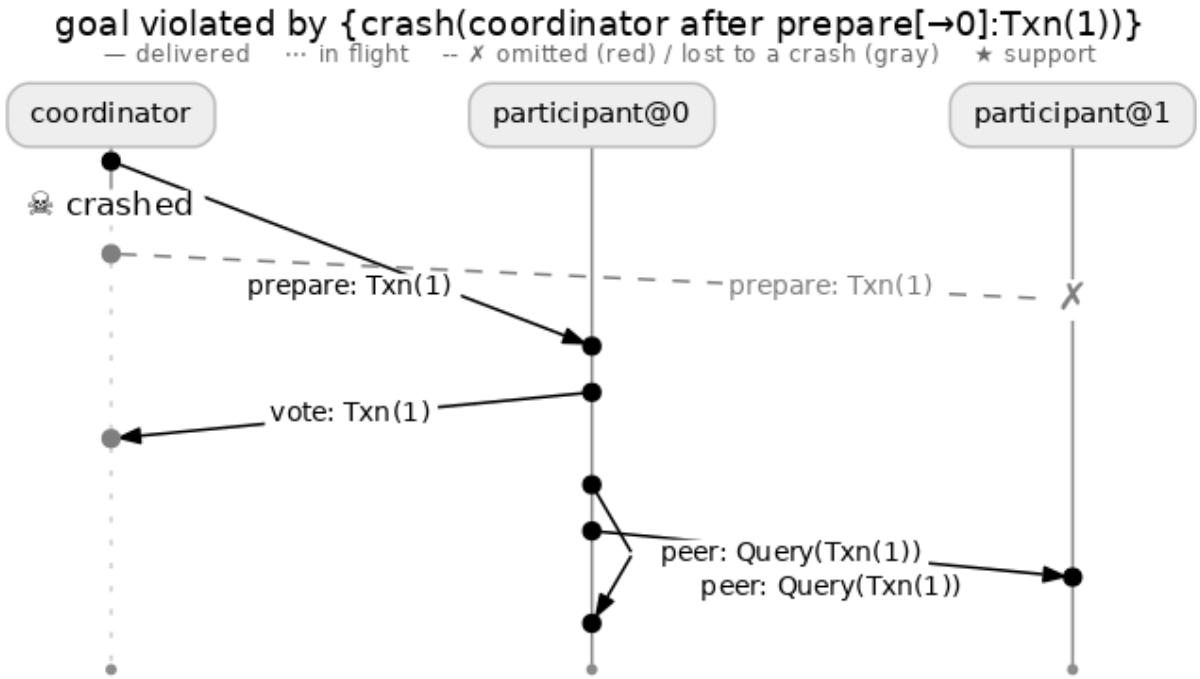
The coordinator may crash

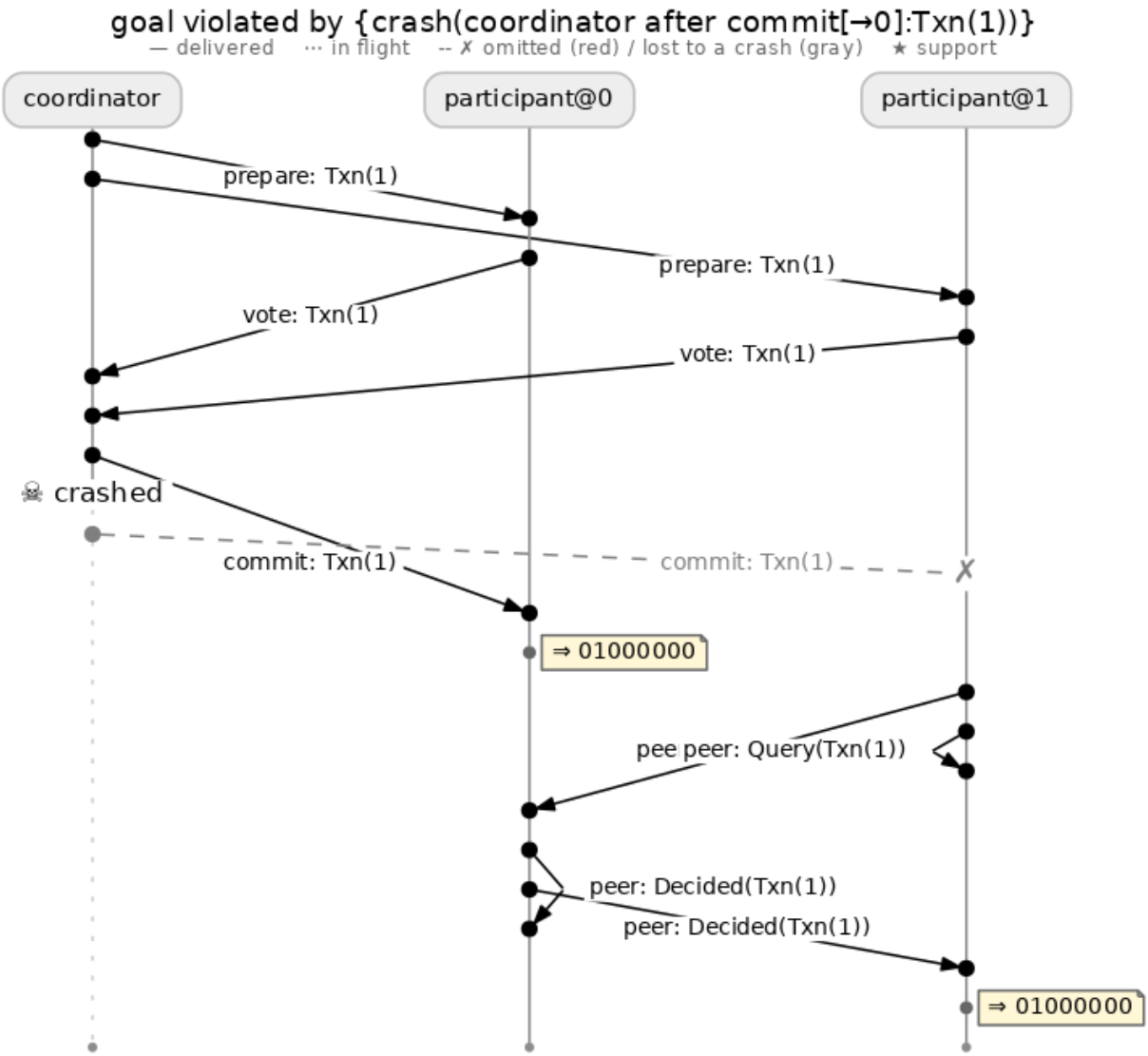
```
cargo test -p hydro_test --lib cluster::two_pc_ldfi::tests::ldfi_all_labels_with_the_harness_fault_model
-- --nocapture --test-threads=1 --exact
```

```
participants (2 members)
  prepares  two_pc_ldfi.rs:86  DIVERGES under crash(coordinator after sending prepare 1 to participant 0)
  commits   :92                DIVERGES under crash(coordinator after sending commit 1 to participant 0)
  peer_in   :98                VACUOUS: the scenario's precondition never held without faults, so nothing
was checked
```

```
  peer      :161              VACUOUS: the scenario's precondition never held without faults, so nothing
was checked

summary: 0 hold, 2 DIVERGE (participants/prepares, participants/commits), 2 vacuous
```





Row	Hydrofrac	Brute force
`prepares`	2 fault sets, 20 interleavings, 0.1 s	4
`commits`	2, 64, 0.4 s	2
`peer_in`, `peer`	vacuous	vacuous

The vacuous rows are about the test, not the protocol. In the fault-free run every participant learns the decision before its timeout, so nobody queries a peer, and there are no facts to check.

The coordinator is held correct

```
cargo test -p hydro_test --lib cluster::two_pc_ldfi::tests::ldfi_all_labels_with_the_coordinator_correct
-- --nocapture --test-threads=1 --exact
```

```
participants (2 members)
  prepares  two_pc_ldfi.rs:86  certified (1 fault set, all schedules)
```

```
  commits   :92                certified (1 fault set, all schedules)
  peer_in   :98                VACUOUS: the scenario's precondition never held without faults, so nothing
was checked
  peer      :161               VACUOUS: the scenario's precondition never held without faults, so nothing
was checked

summary: 2 hold, 2 vacuous
```

No diagram. No fault set is allowed: the coordinator is correct and no participant may crash. The fault-free run is the only one. Cost: 1 fault set and 16 interleavings per row, 0.1 s. Brute force: 1.

3.5.6 ABD register

The ABD replicated register from `hydro_std` (`hydro_std/src/ec_inference_demos/abd.rs`): 3 replicas, 1 client, one write, 8 sampled interleavings. A write has two phases: query the replicas for the latest timestamp, then send the new value to all replicas (phase 2). Each replica keeps the value with the highest timestamp.

```
let outs = abd_register(&replicas, 3 / 2 + 1, writes, reads); // or abd_register_reliable_phase2
let _done = outs.write_done.sim_cluster_output();
let _read = outs.read_result.sim_cluster_output();
let spec = Spec::all_labels(&mut flow);
// .crashable_cluster("replica", &replicas, 0..3).crashable_cluster("client", &clients, 0..1)
// body: w_send.send(0, (1, 10)); quiesce
```

Phase 2 over `broadcast_closed`

```
cargo test -p hydro_std --lib
ec_inference_demos::abd::hydrofrac_tests::all_labels_checks_the_register_at_its_input -- --nocapture --
test-threads=1 --exact
```

```
replicas (3 members)
  phase2_fwd         abd.rs:150  DIVERGES under crash(client@0 after sending phase2 Write{rid: 1, ts:
Ts{…}, value: 10} to replica 1)
  register           :161        DIVERGES under crash(client@0 after sending phase2 Write{rid: 1, ts:
Ts{…}, value: 10} to replica 1)
  queries            :141        DIVERGES under crash(client@0 after sending queries 1 to replica 1)
  phase2             :314        DIVERGES under crash(client@0 after sending phase2 Write{rid: 1, ts:
Ts{…}, value: 10} to replica 1)
  abd_register_with  :165        covered by register: DIVERGES under crash(client@0 after sending phase2
Write{rid: 1, ts: Ts{…}, value: 10} to replica 1)

summary: 0 hold, 5 DIVERGE (replicas/phase2_fwd, replicas/register, replicas/queries, replicas/phase2,
replicas/abd_register_with)
messages per write: {"abd.rs:141": 3, "abd.rs:250": 6, "abd.rs:314": 3}
```

The register (the `Singleton` at `:165`, reported under the function name `abd_register_with`) is covered by its input `register`, the writes taken from phase 2. A client crash partway through phase 2 leaves one replica with the write and the others without it, forever.

No diagram. Cost: 2 fault sets and 16 interleavings per row, about 0.15 s each. Brute force: 2 each, except `queries`: 5.

These are not ABD bugs. ABD promises linearizable reads: a read writes back the value it returns to a majority before returning it, so a later read never sees an older value. ABD's own simulator tests check that promise under crashes (`abd_progress_and_reads_latest_under_replica_crash`, `abd_reads_monotone_under_client_crash`). ABD does not promise that replicas hold the same value. The counterexamples show that `broadcast_closed`'s EC label is conditional on the sender surviving.

Phase 2 over reliable broadcast

`abd_register_reliable_phase2` sends phase 2 over reliable broadcast, so every replica that receives a write echoes it.

```
cargo test -p hydro_std --lib
ec_inference_demos::abd::hydrofrac_tests::all_labels_reliable_phase2_certifies_the_register -- --nocapture
--test-threads=1 --exact
```

```
replicas (3 members)
  phase2_fwd         abd.rs:150  no violation found (20 fault sets, 8 sampled schedules each)
  register           :161        no violation found (22 fault sets, 8 sampled schedules each)
  queries            :141        DIVERGES under crash(client@0 after sending queries 1 to replica 1)
  abd_register_with  :165        covered by register: no violation found
  hops merged into phase2_fwd:
    initial  reliable_broadcast.rs:269  diverges alone under crash(client@0 after sending initial
Write{rid: 1, ts: Ts{…}, value: 10} to replica 1)
    echo     :281                       diverges alone under crash(replica@0 after sending echo (0,
Write{rid: 1, ts: Ts{…}, value: 10}) to replica 1)

summary: 3 hold, 1 DIVERGE (replicas/queries)
messages per write: {"abd.rs:141": 3, "abd.rs:250": 6, "reliable_broadcast.rs:269": 3,
"reliable_broadcast.rs:281": 9}
```

The register now agrees. `queries` still diverges. That costs nothing: a lost query only delays the write.

No diagram. Cost: `phase2_fwd` 20 fault sets, 160 interleavings, 1.5 s; `register` 22, 176, 1.4 s; `queries` 2, 16, 0.2 s. No brute-force measurement.

The price is messages. With 3 replicas, a write sends 21 messages instead of 12: phase 2 is O(n²) instead of O(n).

3.5.7 Uniform agreement

The tests in `hydro_test/src/cluster/uniform_agreement_ldfi.rs` put the generated (non-uniform) check next to a hand-written uniform one, on the same two broadcasts. The source broadcasts items 1 and 2 to three members. Source and members may crash.

The uniform check: every fact that any member delivered, even a member that later crashed, is held by every correct member. Its precondition is that some member delivered something and at most one member crashed:

```
let observed = observed.filter(q!(|(_, fact)| *fact == 1 || *fact == 2));
let delivered = observed.clone().map(q!(|(_, fact)| fact)).unique();
let held = correct.clone().map(q!(|member| (member, ())))
    .join(observed)
    .map(q!(|(member, ((), fact))| (member, fact)))
```

```
    .unique();
spec.pre(delivered.clone().first()
    .zip(correct.clone().count())
    .filter(q!(|(_, correct)| *correct >= 2)));
spec.post(held.count().zip(delivered.count()).zip(correct.count())
    .filter(q!(|((pairs, facts), members)| *pairs == *facts * *members)));
```

The commands (one per row below):

```
cargo test -p hydro_test --lib
cluster::uniform_agreement_ldfi::tests::uniform_agreement_reliable_broadcast_counterexample -- --nocapture
--test-threads=1 --exact
cargo test -p hydro_test --lib
cluster::uniform_agreement_ldfi::tests::uniform_agreement_reliable_broadcast_generated_spec_certifies -- --
nocapture --test-threads=1 --exact
cargo test -p hydro_test --lib cluster::uniform_agreement_ldfi::tests::uniform_agreement_uniform_broadcast
-- --nocapture --test-threads=1 --exact
cargo test -p hydro_test --lib
cluster::uniform_agreement_ldfi::tests::uniform_agreement_uniform_broadcast_generated_spec -- --nocapture
--test-threads=1 --exact
```

```
Reliable under Uniform agreement: 27 hypotheses in 643.8ms
LDFI: goal violated after 27 hypothesis run(s) by {crash(member@0 after reliable_broadcast.rs:156[0→0]:2),
crash(source after reliable_broadcast.rs:130[→0]:2)}

Reliable under Generated agreement: 49 hypotheses in 797.6ms
LDFI: no fault set within bounds violates the goal under any schedule (49 hypothesis run(s))

Uniform(2) under Uniform agreement: 276 hypotheses in 55.2s
LDFI: no fault set within bounds violated the goal in 8 sampled schedule(s) per hypothesis (276 hypothesis
run(s)); not a proof
  certificate RELATIVE to the recorded derivations persisting under the excluded fault sets (hand-written
specification: no soundness claim for the inference)

Uniform(2) under Generated agreement: 105 hypotheses in 5.2s
LDFI: no fault set within bounds violated the goal in 8 sampled schedule(s) per hypothesis (105 hypothesis
run(s)); not a proof
```

Broadcast	**Generated (non-uniform) check**	**Uniform check**
reliable broadcast	certified, 49 fault sets	DIVERGES, 27 fault sets
uniform broadcast, threshold 2	no violation found, 105 fault sets	no violation found, 276 fault sets, 2,208 interleavings

The counterexample: the source sends item 2 to member 0 and crashes. Member 0 delivers item 2, echoes it to itself, and crashes before echoing to anyone else. Members 1 and 2 survive and never get item 2. Under the generated check this passes, because member 0 crashed. Under the uniform check it fails, because member 0 delivered item 2 before crashing.

No diagram. No brute-force measurement.

3.6 Drawing a failing run

The committed Lamport diagrams are under `docs/design/molly-viz-samples/ec/`. Each directory holds `failing.dot` and `failing.png` (one vertical lane per node, time going down; lost messages are dashed lines ending in ✗, and a crash ends a lane with ☠), and `report.md`, the same run as a Mermaid sequence diagram.

To draw a diagram, run a gallery test with `MOLLY_VIZ_OUT` set, then render the `.dot` files with Graphviz:

```
MOLLY_VIZ_OUT=$HOME/gallery cargo test -p hydro_test --lib \
  cluster::ec_ldfi::tests::gallery_ec_broadcast_closed_all_labels -- --test-threads=1 --exact
for f in $HOME/gallery/*/*.dot; do neato -n2 -Tpng -Gdpi=110 "$f" -o "${f%.dot}.png"; done
```

Only that one EC gallery test is committed. To draw another counterexample, copy `gallery_ec_broadcast_closed_all_labels` in `hydro_test/src/cluster/ec_ldfi.rs`: add `.with_traces().with_provenance_graphs()` to the search, pick the counterexample out of the results, and call `ldfi_viz::write_gallery`.

3.7 Lineage compared with brute force

Brute force is `Strategy::Blind` (`.strategy(Strategy::Blind)`). It uses the same fault vocabulary, the same crash limits, and the same loop. It tries untried fault sets smallest first, without lineage. It is not naive: it only considers faults on messages some run actually sent, and it skips fault sets that contain one already found vacuous.

The comparison was measured once per test, before the most recent changes (input coverage, sent-set crashes, the report format, and a change to the order of the closed fan-out). Some lineage counts have shifted since. For example, the reliable-broadcast certificate went from 36 to 49 fault sets after the fan-out change. The tables in section 3.5 give both numbers where they differ. The brute-force counts were not re-measured.

Verdict	Searches	Fault sets (lineage / brute force)	Search time (lineage / brute force)	Lineage fewer / brute force fewer / equal
counterexample	25	114 / 302	21 s / 48 s	12 / 7 / 6
certificate or no violation found	18	475 / 1,012	53 s / 142 s	16 / 0 / 2
vacuous	4	0 / 0	0.2 s / 0.2 s	0 / 0 / 4
total	**47**	**589 / 1,314 (2.2×)**	**74 s / 191 s (2.6×)**	**28 / 7 / 12**

- **Verdicts.** The two strategies agreed on all 47 searches. On all 25 counterexamples they found the same fault set.
- **Certificates.** Lineage never needed more fault sets than brute force, which must try them all. Reliable broadcast 36 vs 115, G-Set 13 vs 40, ISR `writes_at_replicas` 4 vs 68, multi-Paxos `learned` 25 vs 52.
- **Larger programs.** In multi-Paxos, the faults that break a collection are a small part of a large space. Lineage points at them: 5 vs 50 for `prepares`, 105 vs 363 over all labels.
- **Tiny programs.** On 7 small single-hop counterexamples, brute force won 2 fault sets to 5. The breaking source crash comes early in its order. Lineage first crashes each member, which cannot break agreement.
- **Little to prune.** Uniform broadcast: 105 vs 115. Nearly every delivery feeds the result.

- **Time.** A lineage run costs more than a brute-force run, because it computes lineage and calls the solver. Whenever lineage needed fewer fault sets it was also faster, with one exception (the echo broadcast in section 3.5.2).

Most of hydrofrac's value comes from the generated check, the fault model, and the simulator. Lineage adds a moderate speedup, larger on certificates, and it explains each result: every passing run comes with the derivation that made it pass.

4. Limitations and open questions

- **Simulator only.** The fault filter exists only in the simulator. Nothing is injected into deployed programs.
- **One configuration per test.** A certificate covers the test's cluster sizes, inputs, recovery window, and crash limits. It says nothing about others.
- **Sampled interleavings are not proofs.** Multi-Paxos, ISR, G-Set gossip, uniform broadcast, and ABD sample interleavings, because they have too many to enumerate and have non-deterministic operators upstream. Their passing results are "no violation found".
- **Finite runs.** The check runs when the test body ends. Hydrofrac cannot tell "never converges" from "has not converged yet". The body must give the protocol enough time after `end_of_finite_failures()`.
- **What the generated check leaves out.**
- Validity, integrity, and uniformity (section 2.4).
- Derived values, beyond scalars covered by their input. A labelled `Singleton` that is not a fold of a labelled stream is not checked.
- Collections inside a tick, and collections with only local inputs.
- Correctness of `manual_proof!` assertions.

- Observed element types must be `Clone + Serialize`. Exclude others with `Spec::all_labels_except`.

- **Limits of sent-set crashes.** They cannot let through a message first sent under a later fault set, or cut a byte-identical retransmission (section 2.2). `Auto` uses them only where the static check finds no non-deterministic operator upstream.

- **Fault vocabulary.** Omissions and crashes only. No delay beyond reordering, no duplication, no Byzantine behavior. A partition can be written as a set of omissions on lossy channels. Because a message is identified by its bytes, "the first copy was lost and an identical retry got through" cannot be expressed; a protocol whose retries must be distinguished needs distinguishing data in its messages.
- **Cost.** With no crash limit, the number of allowed fault sets grows quickly with the number of crashable nodes. Multi-Paxos, with seven crashable nodes, needs `.crash_budget(1)` to finish.
- **Gaps in precise lineage.** The provenance rewrite does not support every operator (`leader_merge` is compiled without it). Hydrofrac then replays nodes, or falls back to execution-level lineage. This is slower, never wrong.
- **What does the EC label promise?** If it means "same inputs, same result", it is correct for `broadcast_closed` over `fail_stop`, and hydrofrac checks the "same inputs" premise. If it means "replicas converge" unconditionally, the label is wrong whenever a sender can crash partway through a fan-out, as every one-hop example above shows. The Hydro team has not yet said which reading is official.

- **ABD.** Its EC counterexamples are not ABD bugs (section 3.5.6). ABD promises linearizable reads, which it gets by write-back. They show that `broadcast_closed`'s EC is conditional. Making phase 2 reliable makes the register EC, at 21 instead of 12 messages per write with 3 replicas.